VERIQTA

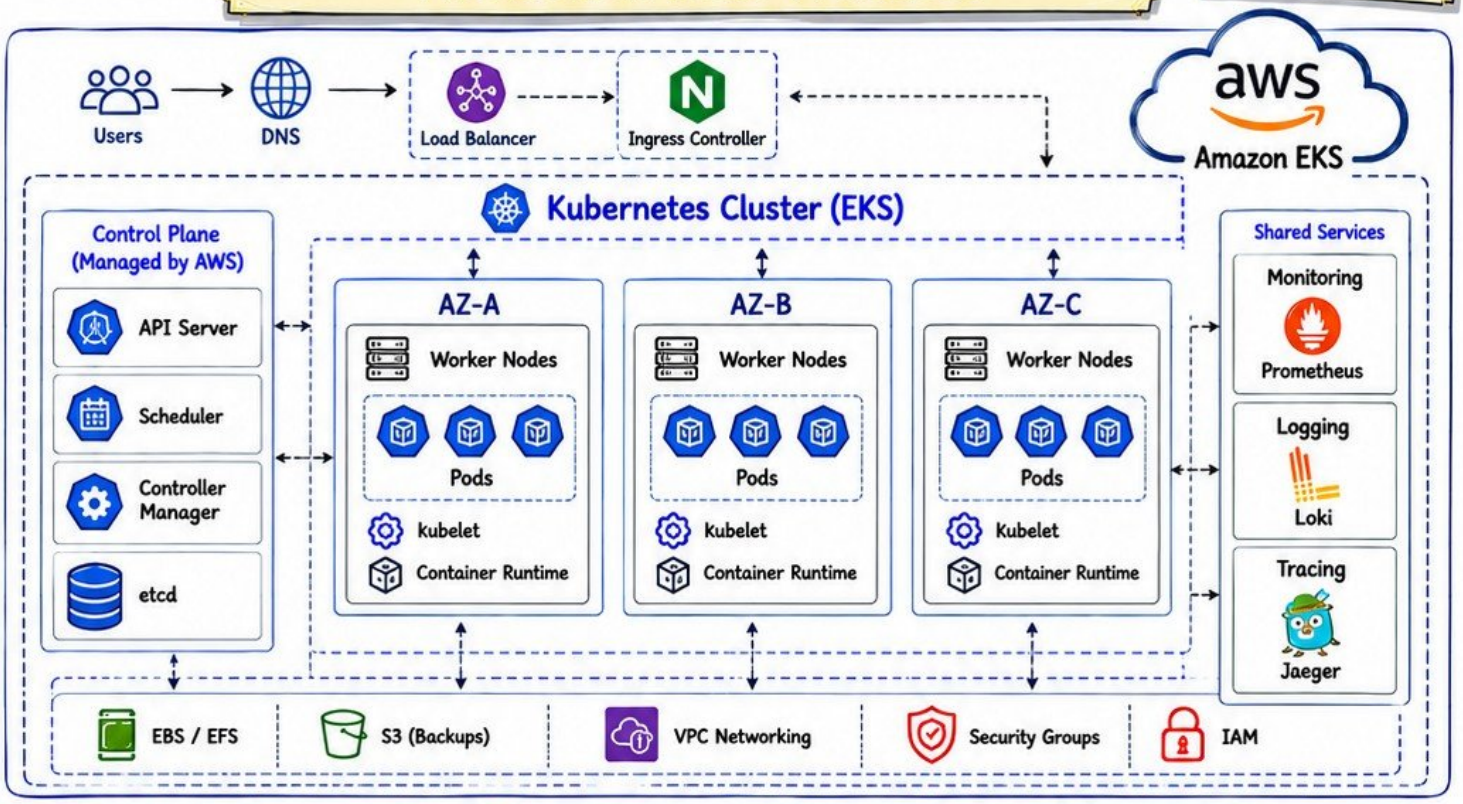
KUBERNETES PRODUCTION INCIDENTS

VERIQTA

VOLUME 1

Real-World Failures, Investigations & Production Best Practices

- ✓ Real Incidents
- ✓ Deep Investigations
- ✓ Root Cause Analysis
- ✓ Fixes & Prevention
- ✓ Production Ready



WHAT YOU'LL LEARN

- Diagnose real production issues
- Follow structured investigation steps
- Apply proven fixes & workarounds
- Adopt best practices for prevention

KEY TOPICS

- Pods, Deployments & Services
- Networking & Ingress
- Storage & Configuration
- Scaling & Performance
- Security & Access Control

BUILT FOR

- DevOps Engineers
- Site Reliability Engineers (SREs)
- Platform Engineers
- Cloud Engineers
- Engineering Managers

VERIQTA

Learn • Build • Troubleshoot • Engineer • Excel

YouTube: VERIQTA | GitHub: veriqta | LinkedIn: VERIQTA | X: @veriqta | Instagram: @veriqta | Telegram: @veriqta_academy

kubernetes

VERIQTA

INCIDENT #1
POD STUCK IN PENDING

The scheduler cannot place the Pod on any node.

AT A GLANCE

- Understand why Pod is Pending
- Investigate scheduler events
- Check resources, taints, affinity
- Validate node conditions
- Fix & verify scheduling

WHAT DOES IT MEAN?

A Pod in Pending state means the scheduler has not been able to find a suitable node to run the Pod yet.

STATUS FLOW

Pending → Running → Succeeded/ Failed

EXAMPLE: POD STATUS

```
$ kubectl get pods myapp -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	NODE
myapp	0/1	Pending	0	2m	<none>

COMMON CAUSES

- Insufficient CPU/Memory
- Node(s) have taints that Pod doesn't tolerate
- NodeSelector / Affinity mismatch
- PV/PVC not available
- Node not Ready
- PodTopologySpreadConstraints cannot be satisfied

1. CHECK EVENTS (FIRST STEP)

```
$ kubectl describe pod myapp
...
Events:
```

Type	Reason	Age	From	Message
Warning	FailedScheduling	2m	default-scheduler	0/3 nodes are available: 3 Insufficient cpu.
Warning	FailedScheduling	2m	default-scheduler	0/3 nodes had taint {dedicated=gpu:NoSchedule}.

2. SCHEDULER DECISIONS

The scheduler evaluates all nodes and filters out those that do not fit the Pod based on:

- ✓ Resource requests (CPU, Memory, Ephemeral Storage)
- ✓ Taints and tolerations
- ✓ NodeSelector / NodeAffinity
- ✓ Pod affinity/anti-affinity rules
- ✓ Pod topology spread constraints
- ✓ Volume constraints (e.g. PVC not bound)
- ✓ Node conditions (Ready = True)

3. RESOURCE REQUESTS INVESTIGATION

Check Pod Requests:

```
$ kubectl get pod myapp -o yaml | grep -A 6 resources:
resources:
  requests:
    cpu: "500m"
    memory: "1Gi"
  limits:
    cpu: "1"
    memory: "2Gi"
```

Scheduler uses REQUESTS to make decisions.

High requests may not fit any node.

Check Node Allocatable:

```
$ kubectl describe node <node-name> | grep -A 10 Allocatable
Allocatable:
  cpu:                1900m
  memory:             3812956Ki
  pods:               110
  ephemeral-storage:  123Gi
```

If total requested resources > allocatable resources across all nodes → Pod stays Pending.

4. TAINTS & TOLERATIONS

Check Node Taints:

```
$ kubectl describe node <node-name> | grep -A 5 Taints
Taints:                 Effect
  dedicated=gpu:NoSchedule
  maintenance=true:NoExecute
```

Pod Tolerations (from pod spec):

```
tolerations:
- key: "dedicated"
  operator: "Equal"
  value: "gpu"
  effect: "NoSchedule"
```

Pod is filtered out if it does not tolerate the taint.

5. NODE CAPACITY INVESTIGATION

Check Node Status:

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
node1	Ready	<none>	10d	v1.29.3
node2	Ready	<none>	10d	v1.29.3
node3	NotReady	<none>	10d	v1.29.3

Describe Node Conditions:

```
$ kubectl describe node <node-name> | grep -A 8 Conditions
Conditions:
```

Type	Status	Reason
Ready	False	KubeletNotReady
MemoryPressure	False	KubeletHasSufficientMemory
DiskPressure	False	KubeletHasNoDiskPressure
PIDPressure	False	KubeletHasSufficientPID

If node is NotReady → scheduler will avoid it.

Check Cluster Resource Pressure:

```
$ kubectl top nodes
```

NAME	CPU(cores)	CPU%	MEMORY(bytes)	MEMORY%
node1	650m	34%	1.2Gi	32%
node2	850m	44%	2.1Gi	55%
node3	–	–	–	–

Identify Insufficient Resources:

- All nodes are at capacity.
- Not enough free CPU/Memory.
- Pod requests exceed available capacity.

6. QUICK FIX APPROACH

1. Reduce resource requests if too high.
2. Add tolerations to match taints.
3. Relax nodeSelector / affinity rules.
4. Add more nodes or increase node size.
5. Fix node conditions (NotReady nodes).
6. Verify PVC bound and storage available.
7. Review PodTopologySpreadConstraints.

USEFUL COMMANDS CHEAT SHEET

```
kubectl get pods -A
kubectl describe pod <pod-name>
kubectl get events --sort-by='.lastTimestamp'
kubectl get nodes
kubectl describe node <node-name>
kubectl get pods <pod-name> -o wide
kubectl top nodes
kubectl get pod <pod-name> -o yaml
```

INVESTIGATION WORKFLOW

Check Events → Analyze Scheduler Reason → Check Resources & Nodes → Check Taints, Affinity & Constraints → Apply Fix & Verify Pod Scheduled

KEY TAKEAWAY

A Pending Pod is the scheduler protecting your cluster from running a workload it cannot support. Follow the evidence, not guesswork.

kubernetes

VERIQTA

INCIDENT #2 CRASHLOOPBACKOFF

The container starts, crashes, and Kubernetes keeps restarting it.

AT A GLANCE

- Understand CrashLoopBackOff
- Investigate root cause
- Check logs, configs, probes
- Fix & validate
- Prevent recurrence

WHAT DOES IT MEAN?

The container is crashing after it starts. Kubernetes is restarting it repeatedly with exponential backoff.

STATUS FLOW

Running → Crashed → Restarting → BackOff (CrashLoop)

EXAMPLE: POD STATUS

```
$ kubectl get pods myapp -o wide

NAME     READY   STATUS             RESTARTS      AGE   NODE
myapp-7d7b9f666c-abc12   CrashLoopBackOff   12 (2m ago)   6m    node1
```

RESTARTS keep increasing and the Pod never becomes Ready.

COMMON CAUSES

- Application errors or exceptions
- Misconfiguration (env, config files)
- Missing dependencies or services
- Database connection failures
- Startup/initialization time too long
- Probe misconfiguration
- Insufficient resources (OOM / CPU)
- Port already in use
- Code bugs introduced in new release

1. CHECK POD STATUS & EVENTS

```
$ kubectl describe pod myapp-7d7b9f6d6c-abc12
...
State:          Waiting
Reason:         CrashLoopBackOff
Last State:     Terminated
  Reason:       Error
  Exit Code:    1
  Started:      Fri, 24 May 2025 10:12:45 +0000
  Finished:     Fri, 24 May 2025 10:12:46 +0000
...
Events:
Type     Reason     Age   Message
Warning  BackOff    2m    Back-off restarting failed container
Normal   Pulled     2m    Container image "myapp:v1.2.0" already present
Normal   Created    2m    Created container myapp
Normal   Started    2m    Started container myapp
Warning  BackOff    2m    Back-off restarting failed container
```

Focus on: Reason, Exit Code, and Events

2. CHECK CONTAINER LOGS

```
$ kubectl logs myapp-7d7b9f6d6c-abc12
or (previous container logs)
$ kubectl logs myapp-7d7b9f6d6c-abc12 --previous
Sample Output:
2025-05-24 10:12:45 INFO  Starting application...
2025-05-24 10:12:45 ERROR Failed to connect to database: connection refused
2025-05-24 10:12:45 FATAL Application exiting
...
```

Look for:

- ✓ Errors / Exceptions
- ✓ Missing files or configs
- ✓ Connection failures
- ✓ Permission denied
- ✓ Out of memory
- ✓ Syntax or runtime errors

3. CHECK PROBES

```
$ kubectl describe pod myapp-...\
Look at Liveness / Readiness probe:
  Liveness:       http-get http://:8080/health
  Initial Delay:  10s   Timeout: 2s   Period: 10s
  Readiness:      http-get http://:8080/ready
  Initial Delay:  5s    Timeout: 2s   Period: 10s
```

Common Probe Issues:

- ✗ Endpoint not ready yet
- ✗ Wrong path or port
- ✗ App takes longer than Initial Delay
- ✗ Probe too aggressive (low timeout/period)
- ✗ App returns 200 but is not actually ready

4. CHECK RESOURCES

```
$ kubectl top pod myapp-7d7b9f6d6c-abc12
$ kubectl describe pod myapp-... | grep -A 5 Limits
```

Watch for:

- High CPU/Memory usage
- OOMKilled (Exit Code 137)
- Limits lower than actual needs

5. CHECK DEPENDENCIES

```
$ kubectl exec -it myapp-... -- sh
# inside the container
```

Check DB:

```
ping db-service
nc -zv db-service 5432
```

Check Config/Secrets:

```
ls -l /etc/config
cat /etc/config/app.yaml
```

6. CHECK PREVIOUS CHANGES

- ☐ Recent code deployment?
- ☐ ConfigMap or Secret updated?
- ☐ Environment variables changed?
- ☐ Database or external service change?
- ☐ New dependencies added?

7. RESOLUTION STEPS

1. Identify root cause from logs & events
2. Fix the issue (code, config, dependencies, probes)
3. Test locally or in staging
4. Deploy the fix
5. Monitor pod status and logs
6. Validate the application is stable

PREVENTION BEST PRACTICES

- Use proper error handling & logging
- Set appropriate resource requests & limits
- Configure probes with realistic timings
- Use startupProbe for slow-start apps
- Implement retries and circuit breakers
- Monitor and alert on CrashLoopBackOff
- Test deployments in staging before prod

KEY TAKEAWAY

CrashLoopBackOff is a symptom, not the root cause. Senior engineers follow the evidence, read the logs, and fix the real problem — not just the symptom.

VERIQTA

CRASHLOOPBACKOFF INVESTIGATION WORKFLOW



Veriqta | veriqta | VERIQTA | @veriqta | @veriqta | @veriqta_academy

kubernetes

VERIQTA

INCIDENT #3 IMAGEPULLBACKOFF

Kubernetes cannot pull the container image from the registry.

AT A GLANCE

- Understand ImagePullBackOff
- Check image, registry & auth
- Verify image tags & repository
- Troubleshoot registry issues
- Fix and prevent recurrence

WHAT DOES IT MEAN?

The kubelet is unable to pull the container image for the Pod. Common reasons include wrong image name/tag, authentication failure, or registry issues.

STATUS FLOW

Pending → Pulling → Failed → ImagePullBackOff

EXAMPLE: POD STATUS

```
$ kubectl get pods myapp -o wide

NAME          READY   STATUS             RESTARTS      AGE   NODE
myapp-5df8c7b7c5-abc12  0/1   ImagePullBackOff  7 (2m ago)   4m    node1
```

Kubelet is retrying to pull the image with exponential backoff.

COMMON CAUSES

- Image name or tag is incorrect
- Image does not exist in registry
- Private registry authentication failure
- Pull secret missiry or incorrect
- Registry is unreachable
- Network or DNS issues
- Rate limiting or permission denied
- Image repository moved or deleted

1. CHECK POD EVENTS

```
$ kubectl describe pod myapp
...
Events:
Type     Reason     Age   From               Message
----     ------     ----  ----               -------
Normal   Scheduled  2m    default-scheduler  Successfully assigned
                                             default/myapp-5df8c7b7c5-abc12
                                             to node1
Warning  Failed     2m    kubelet            Failed to pull image
                                             "myrepo/myapp:1.0.0": rpc error:
                                             code = NotFound desc = failed to
                                             pull and unpack image
Warning  Failed     2m    kubelet            Error: ErrImagePull
Warning  BackOff    1m    kubelet            Back-off pulling image
                                             "myrepo/myapp:1.0.0"
Warning  BackOff    1m    kubelet            Back-off pulling image
```

Focus on the first Failed event for root cause.

2. VERIFY IMAGE NAME & TAG

```
$ kubectl get pod myapp -o yaml | grep image:
  image: myrepo/myapp:1.0.0
```

Checklist:

- ✓ Correct repository path?
- ✓ Correct image name?
- ✓ Correct tag?
- ✓ Image exists in the registry?

Example:

Correct	✓	myrepo/myapp:1.0.0
Wrong Tag	✗	myrepo/myapp:latest
Wrong Repo	✗	myrepo/my-app:1.0.0
Missing Tag	✗	myrepo/myapp:

3. CHECK AUTHENTICATION

```
$ kubectl describe pod myapp
...
Events:
Warning  Failed  1m  kubelet  Failed to pull image
"myrepo/myapp:1.0.0": unauthorized; authentication
required
```

Checklist:

- ✓ Is image from private registry?
- ✓ Does the Pod have imagePullSecrets?
- ✓ Is the secret valid and in the same namespace?

4. CHECK IMAGE IN REGISTRY

```
# Docker Hub Example
$ docker pull myrepo/myapp:1.0.0
Error response from daemon:
manifest for myrepo/myapp:1.0.0
not found

OR

# AWS ECR Example
$ aws ecr describe-images --repository
-name myapp --image-ids imageTag=1.0.0
```

Checklist:

- ✓ Image exists?
- ✓ Correct tag?
- ✓ Repository name is correct?
- ✓ Not expired or deleted?

5. CHECK IMAGEPULLSECRET

```
$ kubectl get pod myapp -o yaml | grep -A2 imagePullSecrets:
imagePullSecrets:
- name: regcred
```

Verify Secret:

```
$ kubectl get secret regcred
$ kubectl describe secret regred
```

Common Issues:

- ✗ Secret missing
- ✗ Wrong secret name
- ✗ Expired credentials
- ✗ Secret in different namespace

6. CHECK REGISTRY CONNECTIVITY

```
$ kubectl exec -it debug --image=busybox --
 nslookup myrepo.com
Server:    10.96.0.10
Address:   10.96.0.10:53
Name:      myrepo.com
Address:   52.84.123.45
```

Checklist:

- ✓ DNS resolution works?
- ✓ Can the node access the registry?
- ✓ Firewall / Security Group blocking?
- ✓ Proxy configured (if required)?

7. CHECK NODE CONFIGURATION

```
$ journalctl -u kubelet -f | grep -i pull
Failed to pull image "myrepo/myapp:1.0.0":
context deadline exceeded
```

Checklist:

- ✓ Kubelet logs for detailed error
- ✓ Container runtime status (containerd/docker)
- ✓ Disk space available?
- ✓ Node has internet access?
- ✓ Correct proxy settings?

8. QUICK FIX CHECKLIST

1. Verify image name & tag
2. Ensure image exists in registry
3. Fix authentication / imagePullSecret
4. Check registry connectivity
5. Restart Pod after fix
6. Monitor Pod until Running

PREVENTION BEST PRACTICES

- Use immutable image tags (e.g., 1.0.0, not latest)
- Store images in reliable private registry (ECR/CCR/ACR)
- Use imagePullSecrets for private registries
- Automate image scanning & validation in CI/CD
- Limit external dependencies in production
- Monitor registry availability and performance

TROUBLESHOOTING WORKFLOW

Check Events → Verify Image & Tag → Check Auth → Check Registry & Connectivity → Apply Fix → Pod Running

KEY TAKEAWAY

ImagePullBackOff means Kubernetes cannot retrieve the image. Follow the evidence, check image, auth, registry, and connectivity — fix the root cause, not just the symptom.

VERIQTA

USEFUL COMMANDS

```
# Pod details
kubectl describe pod <pod-name>

# Get events
kubectl get events --sort-by=.lastTimestamp

# Pod YAML (check image & secrets)
kubectl get pod <pod-name> -o yaml

# Check secret
kubectl get secret <secret-name>

# Test image pull (from node)
ctr -n k8s.io images pull myrepo/myapp:1.0.0

# Check kubelet logs
journalctl -u kubelet -f

# Check nodes
kubectl get nodes

# Exec into debug pod
kubectl run debug --image=busybox -it --rm --restart=Never -- sh

# Check registry (ECR example)
aws ecr describe-repositories
```

VERIQTA

kubernetes

INCIDENT #4
DEPLOYMENT ROLLOUT FAILED

The Deployment was unable to progress the rollout to the desired state.

AT A GLANCE

- Understand why rollouts fail
- Investigate ReplicaSets
- Analyze rolling updates
- Review revision history
- Perform safe rollbacks

WHAT DOES IT MEAN?

A Deployment rollout failed when Kubernetes cannot update Pods to the new version successfully. This can be due to Pod failures, readiness issues, resource limits, or misconfiguration.

STATUS FLOW

Progressing → Progressing (Degraded) → Failed

EXAMPLE: DEPLOYMENT STATUS

```
$ kubectl get deployment myapp
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE	STATUS
myapp	2/5	5	2	15m	Progressing (Failed)

Deployment is stuck and cannot complete the rollout.

COMMON CAUSES

- New Pod crashes or not Ready
- Readiness/Liveness probe failures
- ImagePullBackOff
- Insufficient resources (CPU/Memory)
- Configuration or env errors
- Service dependency unavailable
- Volume mount failures
- Network or DNS issues

1. CHECK DEPLOYMENT EVENTS

```
$ kubectl describe deployment myapp
...
Events:
Type     Reason          Age  From                   Message
----     ------          ---- ----                   -------
Warning  FailedCreate    2m   replicaset-controller  Error creating: pods "myapp-7f7xxxx" is forbidden: exceeded quota
Warning  Unhealthy       1m   kubelet                Readiness probe failed: HTTP probe failed with statuscode: 503
```

Look for FailedCreate, Unhealthy, or BackOff events.

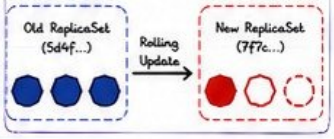
2. CHECK REPLICASETS

```
$ kubectl get rs -l app=myapp
```

NAME	DESIRED	CURRENT	READY	AGE
myapp-7f7c8d5b6c	5	2	2	10m
myapp-5d4f9c7d8b	3	3	3	35m

Key Points:

- ✓ New ReplicaSet (7f7c...) is the one rolling out.
- ✓ Old ReplicaSet (5d4f...) still running.



3. CHECK PODS

```
$ kubectl get pods -l app=myapp
```

NAME	READY	STATUS	RESTARTS	AGE
myapp-7f7c8d5b6c-abc12	0/1	CrashLoopBackOff	5	2m
myapp-7f7c8d5b6c-def34	0/1	ImagePullBackOff	0	2m
myapp-5d4f9c7d8b-ghi56	1/1	Running	0	35m

Check for:

- ✗ CrashLoopBackOff
- ✗ ImagePullBackOff
- ✗ Pending
- ✗ Readiness probe failures

4. CHECK LOGS

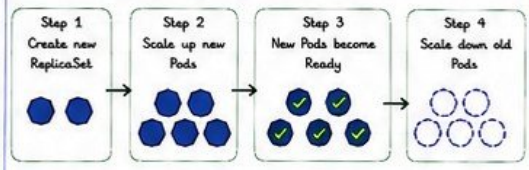
```
$ kubectl logs myapp-7f7c8d5b6c-abc12
...
Error: Failed to connect to database at db-service:5432
ExitCode: 1
```

Or Previous Logs:

```
$ kubectl logs myapp-7f7c8d5b6c-abc12 --previous
```

Investigate application errors, missing configs, dependencies, or connectivity issues.

5. ROLLING UPDATE MECHANICS



If any step fails (e.g., Pods not Ready), the rollout is marked as Failed.

Kubernetes will stop progressing and keep the previous version running.

6. REVISION HISTORY

```
$ kubectl rollout history deployment/myapp
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
3         kubectl set image deployment/myapp myapp=1.2.0
4         kubectl set image deployment/myapp myapp=1.3.0 ← current
```

View details of a revision:

```
$ kubectl rollout history deployment/myapp --revision=3
```

7. ROLLBACK STRATEGY

```
# Rollback to previous revision
$ kubectl rollout undo deployment/myapp

# Rollback to specific revision
$ kubectl rollout undo deployment/myapp --to-revision=3
```

After rollback:

- ✓ Kubernetes scales down bad ReplicaSet
- ✓ Scales up the previous good ReplicaSet
- ✓ Restores stable version

8. QUICK CHECKLIST

- ☑ Check Deployment status and events
- ☑ Inspect new ReplicaSet and Pods
- ☑ Review Pod logs and previous logs
- ☑ Validate probes and configurations
- ☑ Confirm image availability and tags
- ☑ Check resources and quotas
- ☑ Review dependencies and services
- ☑ Rollback if needed and stabilize

ROLLBACK WORKFLOW

Detect Failed Rollout → Analyze Root Cause → Decide Rollback Strategy → Execute Rollback → Verify Service Stability

REMEMBER

Rolling updates are powerful but can fail at any step.
Senior engineers investigate the full chain: events → pods → logs → dependencies → rollback.

KEY TAKEAWAY

A failed rollout does not always mean Kubernetes is broken. It means the new version is unhealthy or misconfigured. Your job is to investigate quickly, protect users by rolling back safely, and ensure the next rollout is reliable.

VERIQTA

USEFUL COMMANDS

$ kubectl get deployment	Check deployment status
$ kubectl describe deployment	Detailed deployment events
$ kubectl get rs	List ReplicaSets
$ kubectl get pods -l app=<app>	List Pods by label
$ kubectl describe pod <pod>	Describe Pod events
$ kubectl logs <pod>	View Pod logs
$ kubectl logs <pod> --previous	View previous container logs
$ kubectl rollout history deployment/<name>	View rollout history
$ kubectl rollout undo deployment/<name>	Rollback to previous revision
$ kubectl rollout undo deployment/<name> --to-revision=N	Rollback to specific revision

VERIQTA

kubernetes

INCIDENT #5
SERVICE CANNOT REACH PODS

The Service exists, but traffic is not reaching the Pods.

AT A GLANCE

- Validate Service selectors
- Inspect Endpoints
- Verify Labels
- Investigate kube-proxy
- Confirm network path

WHAT DOES IT MEAN?

The Service is unable to route traffic to any of its Pods. This usually happens when there is a mismatch in selectors, labels, or when the Endpoints object has no addresses.

STATUS FLOW

Service OK → No Endpoints → Traffic Fails

EXAMPLE: SERVICE STATUS

```
$ kubectl get svc myapp-service
```

NAME	TYPE	CLUSTER-IP	PORT(S)	AGE
myapp-service	ClusterIP	10.96.25.80	80/TCP	5m

Service exists, but application is not accessible.

COMMON CAUSES

- Service selector does not match Pod labels
- Endpoints object has no addresses
- Pods are not Ready
- Pods are in different namespace
- NetworkPolicy blocking traffic
- kube-proxy not updating rules
- Pod port does not match Service targetPort
- Pods crashing or restarting

1. CHECK SERVICE SELECTORS

```
$ kubectl get svc myapp-service -o yaml
...
spec:
  selector:
    app: myapp
    tier: frontend
```

Checklist:

- ✓ Do the selectors match the Pod labels?
- ✓ Any typo in key or value?
- ✓ Are all required labels present?

2. CHECK ENDPOINTS

```
$ kubectl get endpoints myapp-service
```

NAME	ENDPOINTS	AGE
myapp-service	<none>	3m

What it means:

- ✗ <none> = No Pods matched the Service selector or no Ready Pods.
- ✓ IPs listed = Service has Pods to route traffic to.

If <none> is shown:

- Check Service selectors
- Check Pod labels
- Check Pod readiness
- Check namespace

3. CHECK POD LABELS

```
$ kubectl get pods --show-labels
```

NAME	LABELS
myapp-7c9b6d6f-abc12	app=myapp,tier=frontend
myapp-7c9b6d6f-def34	app=myapp,tier=frontend
myapp-7c9b6d6f-ghi56	app=myapp,tier=backend

Checklist:

- ✓ Do Pod labels match Service selectors?
- ✓ Are Pods using correct label keys and values?
- ✓ Are labels missing or incorrect?

4. CHECK POD READINESS

```
$ kubectl get pods
```

NAME	READY	STATUS
myapp-7c9b6d6f-abc12	1/1	Running
myapp-7c9b6d6f-def34	0/1	Not Ready
myapp-7c9b6d6f-ghi56	1/1	Running

Checklist:

- ✓ Are Pods Ready?
- ✓ Check readinessProbe and livenessProbe
- ✓ Review Pod events for failures

5. CHECK PORTS

```
$ kubectl get svc myapp-service -o yaml
ports:
  - port: 80
    targetPort: 8080
```

Checklist:

- ✓ Does targetPort match the containerPort?
- ✓ Is the container listening on the correct port?
- ✓ Is the port exposed in the Pod spec?

6. CHECK NAMESPACE

```
$ kubectl get svc myapp-service -n default
$ kubectl get pods -n default
...
```

Checklist:

- ✓ Are Service and Pods in the same namespace?
- ✓ Cross-namespace Services do not work by default.

7. CHECK NETWORKPOLICY

```
$ kubectl get networkpolicy -n default
$ kubectl describe networkpolicy
```

Checklist:

- ✓ Is there a NetworkPolicy blocking traffic to Pods?
- ✓ Check ingress rules carefully.
- ✓ Temporarily allow all to test.

8. KUBE-PROXY INVESTIGATION

```
$ kubectl -n kube-system get pods -l k8s-app=kube-proxy
$ kubectl -n kube-system logs -l k8s-app=kube-proxy
```

Checklist:

- ✓ Are kube-proxy Pods Running?
- ✓ Check kube-proxy logs for errors.
- ✓ Ensure iptables/IPVS rules are syncing.

TRAFFIC FLOW (EXPECTED)

Client → Service (ClusterIP) → Endpoints (Pod IPs) → Pods

If any step fails, traffic stops.

TROUBLESHOOTING WORKFLOW

Check Service Selectors → Check Endpoints → Verify Labels → Check Pod Readiness → Check NetworkPolicy → Check kube-proxy → Test Service Access

KEY TAKEAWAY

Services cannot reach Pods when Kubernetes cannot find healthy Pods that match the selectors. Always validate selectors, Endpoints, labels, readiness, and networking in this order.

VERIQTA

USEFUL COMMANDS

```
$ kubectl get svc
$ kubectl describe svc <svc-name>
$ kubectl get endpoints <svc-name>
$ kubectl describe endpoints <svc-name>
$ kubectl get pods --show-labels
$ kubectl describe pod <pod-name>
$ kubectl get networkpolicy -A
$ kubectl -n kube-system get pods -l k8s-app=kube-proxy
$ kubectl -n kube-system logs -l k8s-app=kube-proxy
```

PREVENTION BEST PRACTICES

- Use consistent and meaningful labels.
- Validate Services after every deployment.
- Use readinessProbes to ensure healthy traffic routing.
- Document Service and label standards.
- Monitor Endpoints and Service health.

VERIQTA

INCIDENT #6

INGRESS RETURNING 404/502 ERRORS

The request reaches the Ingress, but something breaks before it hits the application.

AT A GLANCE

- Understand why Ingress fails
- Inspect controllers & configs
- Validate backends & services
- Check TLS & certificates
- Fix routing and restore traffic

WHAT DOES IT MEAN?

The Ingress is returning 404 or 502 errors when accessing your application. This can happen due to misconfigured rules, backend issues, TLS problems, or controller errors.

STATUS FLOW



EXAMPLE: INGRESS ERROR

```
$ curl -I https://app.example.com
HTTP/2 404
server: nginx
content-type: text/html
content-length: 146
```

```
$ curl -I https://api.example.com
HTTP/2 502
server: nginx
content-type: text/html
content-length: 157
```

404 = Not Found (routing issue)
502 = Bad Gateway (backend issue)

COMMON CAUSES

- Ingress rules do not match the request
- Incorrect host or path configuration
- Service or Endpoints not found
- Backend Pods not Ready
- Port mismatch (Service ↔ Pod)
- Ingress controller configuration error
- TLS certificate invalid or expired
- Backend timeout or connection refused

1. CHECK INGRESS CONTROLLER

```
$ kubectl get pods -n ingress-nginx
NAME                                        READY   STATUS    RESTARTS   AGE
ingress-nginx-controller-7d4b5f1f9f-abc12   1/1     Running   0          12d
```

Checklist:

- ✓ Controller Pods are Running
- ✓ No CrashLoopBackOff
- ✓ Check Logs for errors

View Logs:

```
$ kubectl logs -n ingress-nginx deploy/ingress-nginx-controller
```

2. CHECK INGRESS RESOURCE

```
$ kubectl get ingress
NAME          CLASS   HOSTS              ADDRESS      PORTS   AGE
app-ingress   nginx   app.example.com    10.0.1.25    80,443  3d
```

Describe Ingress:

```
$ kubectl describe ingress app-ingress
```

Checklist:

- ✓ Rules are correct
- ✓ Host matches the request
- ✓ Paths are defined correctly
- ✓ Backend service name/port is correct

3. CHECK BACKEND SERVICES

```
$ kubectl get svc
NAME          TYPE        CLUSTER-IP     PORT(S)    AGE
app-service   ClusterIP   10.96.25.30    8080/TCP   3d
```

Checklist:

- ✓ Service exists
- ✓ Port matches the application port
- ✓ TargetPort matches the container port
- ✓ Service selector is correct

4. CHECK ENDPOINTS

```
$ kubectl get endpoints app-service
NAME          ENDPOINTS                                   AGE
app-service   10.2.1.15:8080,10.2.2.18:8080               3d
```

Checklist:

- ✓ Endpoints exist
- ✓ IPs are correct
- ✓ Pods behind service are Ready
- ✓ No empty endpoints

5. CHECK PODS BEHIND SERVICE

```
$ kubectl get pods -l app=myapp
NAME                     READY   STATUS             RESTARTS   AGE
myapp-6b8d66f9-abc12     1/1     Running            0          2d
myapp-6b8d8cf9-def34     0/1     CrashLoopBackOff   3          2d
```

Checklist:

- ✓ Pods are Running
- ✓ Readiness probe is passing
- ✓ No CrashLoopBackOff
- ✓ Application is listening on correct port

6. CHECK INGRESS RULES (HOST & PATH)

Example Ingress Rule:

```
rules:
- host: app.example.com
  http:
    paths:
    - path: /
      pathType: Prefix
      backend:
        service:
          name: app-service
          port:
            number: 8080
```

Checklist:

- ✓ Host matches DNS
- ✓ Path matches request
- ✓ PathType is correct (Prefix / Exact)
- ✓ Backend service & port are correct

7. CHECK TLS CONFIGURATION

```
$ kubectl get secret -n default
NAME          TYPE                DATA   AGE
tls-app-cert  kubernetes.io/tls   2      3d
```

Checklist:

- ✓ TLS secret exists
- ✓ Certificate is valid
- ✓ Domain matches certificate (SAN/CN)
- ✓ No certificate expired

8. DEBUG WITH CURL

```
# Check HTTP routing
$ curl -I http://app.example.com

# Check HTTPS & TLS
$ curl -I https://app.example.com

# Check specific path
$ curl -I https://app.example.com/api
```

Checklist:

- ✓ HTTP should redirect to HTTPS
- ✓ HTTPS should return 200 OK
- ✓ Correct path should return 200 OK

9. COMMON ERROR MEANINGS

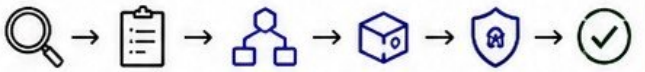
404 Not Found → Routing issue

- Host/path does not match any rule
- Ingress rule misconfiguration
- Wrong pathType

502 Bad Gateway → Backend issue

- No endpoints behind service
- Pods not Ready or crashing
- Port mismatch or connection refused
- Timeout or resource exhaustion

10. INVESTIGATION WORKFLOW



KEY TAKEAWAY

Ingress errors are usually caused by routing rules, backend health, or TLS issues. Follow the traffic path step-by-step, validate each component, and use evidence to isolate the root cause.

VERIQTA

PREVENTION BEST PRACTICES

- Use consistent Ingress naming & annotations
- Automate TLS certificate renewal (Let's Encrypt/ACM)
- Use readiness and liveness probes
- Monitor Ingress controller and backend health
- Test routes after every deployment
- Document routing rules and host/path mappings

INGRESS TRAFFIC FLOW



USEFUL COMMANDS

```
$ kubectl get ingress
$ kubectl describe ingress <name>
$ kubectl get svc
$ kubectl get endpoints <svc-name>
$ kubectl get pods -l <app=name>
$ kubectl logs -n ingress-nginx deploy/ingress-nginx-controller
```

kubernetes

VERIQTA

INCIDENT #7
CONFIGMAP OR SECRET NOT UPDATING

You updated the ConfigMap or Secret, but your Pods are still using old configuration.

AT A GLANCE

- Understand update behavior
- Check mounted volumes
- Check environment variables
- Verify Pod reload behavior
- Fix and apply best practices

WHAT DOES IT MEAN?

You updated a ConfigMap or Secret, but Pods are still using the old values. This happens due to how Kubernetes mounts or injects configuration.

STATUS FLOW

Config Updated → Pods Still Using Old Values → Investigate & Fix → Apply Changes

EXAMPLE: CONFIG UPDATE

```
# Update ConfigMap
$ kubectl edit configmap app-config

# Check Pods (still running with old config)
$ kubectl get pods -l app=myapp
NAME                     READY   STATUS    AGE
myapp-7d8f9f6d6c-abc12   1/1     Running   2d
```

Config updated, but Pod did not reload automatically.

COMMON CAUSES

- Mounted ConfigMap/Secret does not update inside running Pod
- Environment variables are set only at startup
- Application does not reload config
- Pod was not restarted after update
- Using subPath prevents updates
- ConfigMap/Secret name mismatch
- Missing checksum annotation for automatic rollout

1. CHECK CONFIGMAP OR SECRET

```
# Check ConfigMap
$ kubectl get configmap app-config -o yaml

# Check Secret
$ kubectl get secret app-secret -o yaml
```

Checklist:

- ✓ Does the config contain your changes?
- ✓ Is the correct ConfigMap/Secret being updated?
- ✓ Any typos in keys or values?

2. CHECK HOW IT IS CONSUMED

```
# Check volume mounts
$ kubectl describe pod <pod-name> |
  grep -A 5 -i "mounts:"

# Check env vars
$ kubectl describe pod <pod-name> |
  grep -A 5 -i "environment:"
```

Checklist:

- ✓ Mounted as volume or subPath?
- ✓ Injected as environment variables?
- ✓ Is the Pod using the expected ConfigMap/Secret?

3. CHECK MOUNTED CONTENT

```
# Exec into Pod
$ kubectl exec -it <pod-name> -- sh

# Check mounted file
$ cat /etc/config/app.conf

# Check Secret file
$ cat /etc/secret/secret.conf
```

Checklist:

- ✓ Does the file show old values?
- ✓ If yes, the volume has not updated.
- ✓ If mounted with subPath, it will never update.

4. CHECK ENVIRONMENT VARIABLES

```
# Exec into Pod
$ kubectl exec -it <pod-name> -- sh

# Check env var
$ env | grep APP_
```

Checklist:

- ✓ Environment variables are set only at container start.
- ✓ Pods must be restarted to pick new values.

5. CHECK ROLLOUT & PODS

```
# Check Pods
$ kubectl get pods -l app=myapp

# Check rollout history
$ kubectl rollout history deployment/myapp
```

Checklist:

- ✓ Pods still running with old spec?
- ✓ No rollout triggered?
- ✓ Deployment/StatefulSet not updated?

6. FIX: RESTART PODS / ROLLOUT

```
# Restart deployment
$ kubectl rollout restart deployment/myapp

# Or delete Pods
$ kubectl delete pod -l app=myapp
```

Checklist:

- ✓ This forces Pods to reload the new ConfigMap/Secret.
- ✓ For StatefulSets, restart one by one if needed.

7. AUTOMATE WITH CHECKSUM

```
# Add annotation to deployment
metadata:
  annotations:
    checksum/config: {{ include (print
  $.Template.BasePath "/configmap.yaml")
  . | sha256sum }}
```

Checklist:

- ✓ When ConfigMap/Secret changes, checksum changes.
- ✓ This triggers automatic rollout.

8. VERIFY AFTER UPDATE

```
# Check new Pods
$ kubectl get pods -l app=myapp

# Exec and verify
$ kubectl exec -it <pod-name> -- sh
```

Checklist:

- ✓ Pods are using the new values?
- ✓ Application behaves correctly?
- ✓ No errors in logs?

PREVENTION BEST PRACTICES

- Use checksum annotations for automatic rollouts.
- Avoid using subPath for ConfigMap/Secret mounts.
- Design applications to reload config gracefully.
- Document configuration dependencies.
- Use versioned ConfigMaps/Secrets for safety.

CONFIG UPDATE WORKFLOW

1. Update ConfigMap or Secret
2. Verify Changes
3. Trigger Rollout / Restart
4. Verify in Pod
5. Application Using New Config

KEY TAKEAWAY

ConfigMaps and Secrets do not magically update inside running Pods. Understand how they are consumed, restart or rollout your Pods, and automate updates to avoid silent configuration failures.

VERIQTA

USEFUL COMMANDS

- kubectl get configmap -o yaml
- kubectl get secret -o yaml
- kubectl describe pod <pod-name>
- kubectl exec -it <pod-name> -- sh
- kubectl rollout restart deployment/<name>
- kubectl delete pod -l <label>
- kubectl rollout history deployment/<name>
- kubectl logs -f <pod-name>
- kubectl get pods -l <label>
- kubectl diff -f <manifest.yaml>

kubernetes

VERIQTA

INCIDENT #8
PERSISTENTVOLUME CANNOT BE MOUNTED

The Pod is running, but the storage is not accessible.

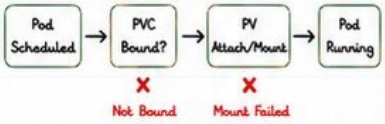
AT A GLANCE

- Validate PV and PVC status
- Check StorageClass & binding
- Inspect CSI driver health
- Review node and mount issues
- Fix and restore storage access

WHAT DOES IT MEAN?

The PersistentVolume (PV) cannot be mounted to the Pod.
This prevents the application from reading or writing data.
Common causes include binding issues, CSI driver problems, permissions, or node/mount failures.

STATUS FLOW



EXAMPLE: POD STATUS

```
$ kubectl get pods -o wide
NAME              READY   STATUS              RESTARTS   AGE   NODE
myapp-7d8f9f6d6c-abc12   0/1   ContainerCreating   0   2m   node1

$ kubectl describe pod myapp-7d8f9f6d6c-abc12
Events:
  Warning  FailedMount  1m (x5 over 2m)  kubelet
  MountVolume.SetUp failed for volume "data":
  rpc error: code = Internal desc =
  failed to mount volume: exit status 32
```

PVC exists, but the volume cannot be attached or mounted.

COMMON CAUSES

- PVC is not bound to any PV
- PV is in Failed or Released state
- StorageClass misconfigured
- CSI driver not healthy or missing
- Node does not have correct permissions
- Volume already attached to another node
- NFS/EBS/EFS mount target unreachable
- Filesystem corruption or full disk
- Wrong access mode (RWO/ROX/RWX)

1. CHECK PVC STATUS

```
$ kubectl get pvc
NAME        STATUS   VOLUME   CAPACITY   ACCESS MODES
myapp-pvc   Bound    pv-001   10Gi       RWO
```

Checklist:

- ✓ Is PVC Bound?
- ✓ If Pending, check Events.
- ✓ Is the requested StorageClass available?

2. CHECK PV STATUS

```
$ kubectl get pv
NAME     STATUS   CAPACITY   ACCESS MODES   CLAIM
pv-001   Bound    10Gi       RWO            default/
                                            myapp-pvc
```

Checklist:

- ✓ Is PV Bound?
- ✓ Is PV in Failed or Released state?
- ✓ Does capacity match the PVC request?

3. CHECK STORAGECLASS

```
$ kubectl get storageclass
NAME                 PROVISIONER             RECLAIMPOLICY
standard (default)   ebs.csi.aws.com         Delete
```

Checklist:

- ✓ Is StorageClass provisioner correct?
- ✓ Are parameters valid?
- ✓ Is the default StorageClass available?

4. CHECK EVENTS

```
$ kubectl get events --sort-by='.lastTimestamp'
LAST SEEN   TYPE      REASON        OBJECT              MESSAGE
1m          Warning   FailedMount   pod/myapp-....      MountVolume.
                                                        SetUp
                                                        failed....
```

Checklist:

- ✓ Look for FailedMount events.
- ✓ Check CSI/Provisioner errors.
- ✓ Review node and mount errors.

5. CHECK CSI DRIVER

```
$ kubectl get csidrivers
NAME              ATTACHREQUIRED   PODINFOONMOUNT   AGE
ebs.csi.aws.com   true             true             10d
```

Checklist:

- ✓ Is CSI driver present?
- ✓ Is it Healthy?
- ✓ Check controller and node plugin pods.

6. CHECK NODE & MOUNT

```
$ kubectl describe node node1
Conditions:
  Type              Status
  Ready             True
  DiskPressure      False
  FileSystemPressure  False
  PIDPressure       False
```

Checklist:

- ✓ Is node Ready?
- ✓ Check disk space on node.
- ✓ Verify mount points and permissions.

7. CHECK VOLUME ATTACHMENT

```
$ kubectl get volumeattachments
NAME                                   ATTACHER   ATTACHED   AGE
csi-1a2b3c4d-5678-90ab-cdef-123456789...
                                       node1      true       2m
```

Checklist:

- ✓ Is volume attached to the correct node?
- ✓ If stuck, try detaching.
- ✓ Ensure no multi-attach conflicts.

8. VERIFY STORAGE ACCESS

```
$ kubectl exec -it myapp-pod -- sh
/$ df -h /data
Filesystem      Size  Used  Avail  Use%  Mounted on
/dev/xvdf       10G   2.0G  8.0G   20%   /data
```

Checklist:

- ✓ Can the pod read/write the mount?
- ✓ Verify application permissions.
- ✓ Confirm filesystem is healthy.

PREVENTION BEST PRACTICES

- Use appropriate StorageClass for your workload.
- Monitor PV and PVC status.
- Enable alerts for FailedMount events.
- Keep CSI drivers updated.
- Validate node disk capacity and permissions.
- Document storage layout and backup strategy.

STORAGE MOUNT WORKFLOW



KEY TAKEAWAY

PersistentVolume mount failures stop your application from accessing data. Always validate the entire storage chain: PVC, PV, StorageClass, CSI driver, and node mount.
Fix the root cause, then verify access.

VERIQTA

USEFUL COMMANDS

- kubectl get pvc
- kubectl describe pvc <pvc-name>
- kubectl get pv
- kubectl describe pv <pv-name>
- kubectl get storageclass
- kubectl describe storageclass <sc-name>
- kubectl get events --sort-by='.lastTimestamp'
- kubectl describe pod <pod-name>
- kubectl get csidrivers
- kubectl get volumeattachments
- kubectl describe node <node-name>
- kubectl exec -it <pod-name> -- df -h

VERIQTA

INCIDENT #9

NODE SUDDENLY BECOMES NOTREADY

The Node was healthy, then suddenly shows NotReady.
Pods on the node may be unavailable.

AT A GLANCE

- Identify why Node is NotReady
- Check Kubelet & Node conditions
- Investigate Disk & Memory pressure
- Review system logs and metrics
- Restore Node health and stability

WHAT DOES IT MEAN?

The Kubernetes Node is reporting NotReady status.
This means Kubelet is not able to communicate with the API server or the Node is under resource pressure or experiencing failures.

STATUS FLOW

Node Ready → Node Becomes NotReady → Pods Impacted (Evicted) → Node Ready

Evicted

EXAMPLE: NODE STATUS

```
$ kubectl get nodes
NAME            STATUS     ROLES    AGE   VERSION
ip-10-20-1-45   NotReady   <none>   2d3h  v1.28.4

$ kubectl describe node ip-10-20-1-45 | grep -A10 "Conditions:"
Conditions:
Type             Status  LastHeartbeatTime  LastTransitionTime  Reason                  Message
Ready            False   10s                1m30s               KubeletNotReady         runtime network error
MemoryPressure   True    10s                1m30s               KubeletHasInsufficientMemory
DiskPressure     True    10s                1m20s               NodeHasDiskPressure
PIDPressure      False   10s                2d3h                KubeletHasSufficientPID
NetworkUnavailable False 10s                2d3h                CalicoIsUp
```

Node is NotReady and under resource pressure.

COMMON CAUSES

- Kubelet service stopped or crashed
- DiskPressure (low disk space)
- MemoryPressure (insufficient memory)
- Network issues (Kubelet <-> API server)
- Container runtime failures
- High load causing Node unresponsiveness
- Kernel panic or system crash
- Cloud provider issues (underlying host)

1. CHECK NODE STATUS

```
$ kubectl get nodes -o wide
$ kubectl describe node <node-name>
```

Checklist:
- ✓ Node shows NotReady?
- ✓ Check Conditions section
- ✓ Note the Reason and Message
- ✓ When did it transition?

2. CHECK KUBELET SERVICE

```
$ ssh <node-ip>
$ systemctl status kubelet --no-pager
```

Checklist:
- ✓ Is Kubelet running?
- ✓ Any recent restarts?
- ✓ Check Kubelet logs for errors

3. CHECK DISK PRESSURE

```
$ df -h
$ kubectl describe node <node-name> |
  grep -A2 "DiskPressure"
```

Checklist:
- ✓ Is disk usage high (>85%)?
- ✓ Any partitions full?
- ✓ Check inode usage

4. CHECK MEMORY PRESSURE

```
$ free -h
$ kubectl describe node <node-name> |
  grep -A2 "MemoryPressure"
```

Checklist:
- ✓ Is memory usage high?
- ✓ OOM events in system logs?
- ✓ Swap usage (if enabled)?

5. CHECK NETWORK CONNECTIVITY

```
$ ping <api-server-ip>
$ nc -zv <api-server-ip> 6443
```

Checklist:
- ✓ Can node reach API server?
- ✓ Any packet loss or timeouts?
- ✓ DNS resolution working?

6. CHECK SYSTEM LOAD

```
$ uptime
$ top -b -n 1 | head -20
```

Checklist:
- ✓ Is load average high?
- ✓ CPu usage near 100%?
- ✓ Too many processes running?

7. CHECK CONTAINER RUNTIME

```
$ systemctl status containerd
$ journalctl -u containerd -n 50 --no-pager
```

Checklist:
- ✓ Is container runtime running?
- ✓ Any errors in containerd logs?
- ✓ Can it pull or run containers?

8. CHECK KUBELET LOGS

```
$ journalctl -u kubelet -n 100 --no-pager
```

Checklist:
- ✓ Look for errors or warnings
- ✓ Check for eviction messages
- ✓ Look for network errors

9. CHECK NODE EVENTS

```
$ kubectl get events --field-selector
  involvedObject.kind=Node
  --sort-by='.lastTimestamp"
```

Checklist:
- ✓ Look for Warning events
- ✓ Check KubeletNotReady
- ✓ Check resource pressure events

10. VERIFY POD IMPACT

```
$ kubectl get pods --all-namespaces \
  -field-selector spec.nodeName=<node>
```

Checklist:
- ✓ Which Pods are on this Node?
- ✓ Are they Evicted or Terminating?
- ✓ Which workloads are affected?

11. FIX THE ROOT CAUSE

- ✓ Free disk space or cleanup logs
- ✓ Reduce memory usage or scale down
- ✓ Restart Kubelet service
- ✓ Fix network connectivity
- ✓ Address container runtime issues
- ✓ Reboot Node if necessary

12. VERIFY NODE RECOVERY

```
$ kubectl get nodes
$ kubectl describe node <node-name> |
  grep -A5 "Conditions:"
```

Checklist:
- ✓ Node status is Ready?
- ✓ All conditions are True
- ✓ Pods are running again

NODE CONDITIONS (KEY)

Ready: Node can accept Pods
MemoryPressure: Node is low on memory
DiskPressure: Node is low on disk space
PIDPressure: Node is out of PIDs
NetworkUnavailable: Network plugin issue

RECOVERY WORKFLOW

Detect NotReady → Investigate Conditions → Fix Root Cause → Restore Node Health → Verify & Monitor

KEY TAKEAWAY

A NotReady Node is a warning that something is wrong at the host level. Always check Kubelet, resources, and system health. Fix the root cause, then verify full recovery.

VERIQTA

- $ Monitor Node resources (CPU, Memory, Disk)
- $ Set up alerts for pressure conditions
- $ Keep Kubelet and system updated
- $ Use log rotation and cleanup policies
- $ Use cluster autoscaler for capacity

USEFUL COMMANDS

- kubectl get nodes -o wide
- kubectl describe node <node>
- kubectl get events --sort-by='.lastTimestamp'
- systemctl status kubelet
- journalctl -u kubelet -n 100 --no-pager
- df -h
- free -h
- uptime
- kubectl top nodes
- kubectl get pods --all-namespaces -o wide
- ssh <node-ip>

VERIQTA

INCIDENT #10
COREDNS RESOLUTION FAILURE

Pods and Services cannot resolve DNS names.

AT A GLANCE

- Validate CoreDNS Pods and logs
- Test DNS resolution from Pods
- Check Service discovery
- Review DNS policies and configs
- Fix network or configuration issues

WHAT DOES IT MEAN?

CoreDNS is the DNS server for your Kubernetes cluster.
When it fails, Pods cannot resolve Service names, external domains, or even internal cluster names.
This breaks Service discovery and causes application failures.

STATUS FLOW

DNS Working → DNS Broken → Resolution Fails → Services Down

DNS Broken: ✗ Symptoms Appear
Resolution Fails: ✗ Service Discovery Fails

EXAMPLE: DNS FAILURE

```
$ kubectl get pods -n kube-system -l k8s-app=kube-dns
NAME                       READY   STATUS             RESTARTS   AGE
coredns-6d4b75cb6d-abc12   0/1     CrashLoopBackOff   12         3m
coredns-6d4b75cb6d-def34   0/1     CrashLoopBackOff   9          3m

$ kubectl logs -n kube-system coredns-6d4b75cb6d-abc12
[ERROR] plugin/kubernetes: failed to list *v1.Service: Get
 "https://10.96.0.1:443/api/v1/services?limit=500":
 dial tcp 10.96.0.1:443: i/o timeout
```

CoreDNS is not healthy.
Cluster DNS resolution is not working.

COMMON CAUSES

- CoreDNS Pods are crashing or pending
- API server connectivity issues
- NetworkPolicy blocking DNS traffic
- Incorrect CoreDNS configuration
- Node DNS settings misconfigured
- Service IP range overlap
- kube-proxy or CNI networking issues
- Resource exhaustion (CPU/Memory)
- Upstream DNS or internet issues

1. CHECK COREDNS POD STATUS

```
$ kubectl get pods -n kube-system -l k8s-app=kube-dns
$ kubectl describe pod -n kube-system <coredns-pod>
```

Checklist:
- ✓ Are CoreDNS Pods Running?
- ✓ Any CrashLoopBackOff or Errors?
- ✓ Are resources (CPU/Memory) enough?

2. CHECK COREDNS LOGS

```
$ kubectl logs -n kube-system <coredns-pod>
$ kubectl logs -n kube-system <coredns-pod> -f
```

Checklist:
- ✓ Any errors or timeouts?
- ✓ Failed to reach API server?
- ✓ Plugin errors or panics?

3. TEST DNS FROM POD

```
$ kubectl run dns-test --rm -it --image=busybox
 --restart=Never -- nslookup kubernetes.default
$ kubectl run dns-test --rm -it --image=busybox
 --restart=Never -- nslookup google.com
```

Checklist:
- ✓ Can resolve kubernetes.default?
- ✓ Can resolve other Services?
- ✓ Can resolve external domains?

4. CHECK COREDNS SERVICE

```
$ kubectl get svc -n kube-system kube-dns
$ kubectl describe svc -n kube-system kube-dns
```

Checklist:
- ✓ Does kube-dns Service exist?
- ✓ ClusterIP should be 10.96.0.10
- ✓ Endpoints should have Pod IPs

5. CHECK ENDPOINTS

```
$ kubectl get endpoints -n kube-system kube-dns
$ kubectl describe endpoints kube-dns -n kube-system
```

Checklist:
- ✓ Are endpoints present?
- ✓ Do they point to CoreDNS Pods?
- ✓ Any missing or empty endpoints?

6. CHECK NETWORKPOLICY

```
$ kubectl get networkpolicy -A
$ kubectl describe networkpolicy -n kube-system
```

Checklist:
- ✓ Is CoreDNS blocked?
- ✓ Allow UDP/TCP 53 from Pods
- ✓ Review policies in kube-system

7. CHECK NODE DNS SETTINGS

```
# On each node
$ cat /etc/resolv.conf
$ systemctl status systemd-resolved
```

Checklist:
- ✓ Nameserver reachable?
- ✓ Search domains correct?
- ✓ No wrong upstream DNS?

8. CHECK KUBE-PROXY & CNI

```
$ kubectl -n kube-system get pods -l k8s-app=kube-proxy
$ kubectl -n kube-system get pods -o wide
```

Checklist:
- ✓ Is kube-proxy running?
- ✓ Are CNI Pods healthy?
- ✓ Any networking errors?

9. CHECK API SERVER CONNECTIVITY

```
$ kubectl run apiserver-test --rm -it --image=busybox \
 --restart=Never -- wget -qO- https://kubernetes.default
```

Checklist:
- ✓ Can CoreDNS reach API server?
- ✓ Any TLS or timeout errors?
- ✓ API server reachable?

10. CHECK COREDNS CONFIGMAP

```
$ kubectl -n kube-system get configmap coredns -o yaml
```

Checklist:
- ✓ Is ConfigMap valid?
- ✓ Check forwarders section
- ✓ Look for syntax errors

11. RESTART COREDNS

```
$ kubectl rollout restart deployment coredns \
 -n kube-system
$ kubectl rollout status deployment coredns -n kube-system
```

Checklist:
- ✓ Restart failed Pods
- ✓ Wait for rollout to complete
- ✓ Check Pods become Ready

12. VERIFY RESOLUTION AGAIN

```
$ kubectl run dns-test --rm -it --image=busybox
 --restart=Never -- nslookup kubernetes.default
```

Checklist:
- ✓ DNS resolution works?
- ✓ Service discovery works?
- ✓ Applications can connect?

DNS RESOLUTION FLOW

Pod → CoreDNS (10.96.0.10) → Service Discovery → Cluster IP or External IP

If CoreDNS fails, the chain breaks and Services become unreachable.

DNS TROUBLESHOOTING WORKFLOW

Check CoreDNS Pods → Check Logs → Test DNS From Pod → Check Service & Endpoints → Review NetworkPolicy → Fix & Restart → Verify Resolution

KEY TAKEAWAY

CoreDNS is the backbone of Service discovery in Kubernetes.
When DNS fails, everything fails.
Always validate CoreDNS health, network connectivity, and policies before looking at applications.
Fix DNS first, then verify Services.

VERIQTA

PREVENTION BEST PRACTICES

- ✓ Monitor CoreDNS Pods (CPU, Memory, Restarts)
- ✓ Set resource requests and limits
- ✓ Allow UDP/TCP 53 in NetworkPolicies
- ✓ Use upstream DNS with high availability
- ✓ Backup CoreDNS ConfigMap before changes

USEFUL COMMANDS

- kubectl get pods -n kube-system -l k8s-app=kube-dns
- kubectl logs -n kube-system <coredns-pod> -f
- kubectl run dns-test --rm -it --image=busybox -- nslookup <name>
- kubectl get svc -n kube-system kube-dns
- kubectl get endpoints -n kube-system kube-dns
- kubectl get networkpolicy -A
- kubectl rollout restart deployment coredns -n kube-system
- kubectl -n kube-system get configmap coredns -o yaml
- cat /etc/resolv.conf

VERIQTA

kubernetes

INCIDENT #11

HIGH POD CPU AND MEMORY USAGE

Your Pods are running, but consuming too much CPU or Memory, causing slow performance or crashes.

AT A GLANCE

- Identify high CPU or Memory usage
- Review resource requests & limits
- Investigate OOMKilled events
- Optimize application performance
- Right-size and set best practices

WHAT DOES IT MEAN?

One or more Pods are using excessive CPU or Memory compared to their requests and limits. This can cause slow performance, throttling, or Pod crashes (OOMKilled).

STATUS FLOW

Normal Usage → High Usage → Throttling or OOM → Performance Issue / Crash

High Usage ↓ Investigate

Performance Issue / Crash ↓ Tune & Fix

EXAMPLE: HIGH CPU / MEMORY POD

```
$ kubectl top pods --all-namespaces
NAMESPACE     NAME                        CPU(cores)   MEMORY(bytes)
app           myapp-7c9d6b8f9b-abc12      950m         1.2Gi
app           myapp-7c9d6b8f9b-def34      920m         1.1Gi
monitoring    prometheus-0                200m         512Mi
$ kubectl describe pod myapp-7c9d6b8f9b-abc12
  Last State:  Terminated
  Reason:      OOMKilled
  Exit Code:   137
```

Pod is using too much CPU/Memory or was killed due to Out Of Memory (OOMKilled).

COMMON CAUSES

- Resource requests too low
- Resource limits too high or unset
- Memory leaks in the application
- CPU intensive processes or loops
- Too many concurrent connections
- Improper garbage collection
- Large in-memory caches
- No limits set (neighbor impact)
- Inefficient code or queries

1. FIND TOP CONSUMERS

```
$ kubectl top pods --all-namespaces
$ kubectl top nodes
$ kubectl top pods -n <namespace>
  --sort-by=cpu
$ kubectl top pods -n <namespace>
  --sort-by=memory
```

Checklist:

- ✓ Identify Pods using highest CPU
- ✓ Identify Pods using highest Memory
- ✓ Note namespaces and Pod names

2. CHECK POD RESOURCE SPECS

```
$ kubectl describe pod <pod-name> -n <ns> \
| grep -A5 -E "Requests|Limits|cpu|memory"
$ kubectl get pod <pod-name> -n <ns> -oyaml \
| grep -A5 resources:
```

Checklist:

- ✓ Review CPU requests and limits
- ✓ Review Memory requests and limits
- ✓ Compare with actual usage

3. CHECK FOR OOMKILLED

```
$ kubectl describe pod <pod-name> -n <ns> \
| grep -i -A3 "Last State"
$ kubectl get pod <pod-name> -n <ns>
$ kubectl get events -n <ns> \
  --sort-by='.lastTimestamp'
```

Checklist:

- ✓ Look for OOMKilled in Last State
- ✓ Check Exit Code 137
- ✓ Check recent OOM events

4. ANALYZE CPU THROTTLING

```
$ kubectl describe pod <pod-name> -n <ns> \
| grep -i throttled -A2
$ kubectl exec <pod-name> -n <ns> -- \
  cat /sys/fs/cgroup/cpu.stat
```

Checklist:

- ✓ Check if CPU throttling is happening
- ✓ High throttled time means limit too low
- ✓ Application may be CPU bound

5. CHECK MEMORY USAGE TREND

```
$ kubectl exec <pod-name> -n <ns> -- \
  free -h
$ kubectl exec <pod-name> -n <ns> -- \
  ps aux --sort=-%mem | head
```

Checklist:

- ✓ Check memory usage inside Pod
- ✓ Identify top memory consuming processes
- ✓ Look for memory leak patterns

6. CHECK HPA / VPA (IF USED)

```
$ kubectl get hpa -n <ns>
$ kubectl describe hpa <hpa-name> -n <ns>
$ kubectl get vpa -n <ns>
$ kubectl describe vpa <vpa-name> -n <ns>
```

Checklist:

- ✓ Is HPA scaling based on metrics?
- ✓ Are metrics thresholds correct?
- ✓ Review VPA recommendations

7. CHECK APPLICATION METRICS

```
$ kubectl exec <pod-name> -n <ns> -- \
  curl -s http://localhost:8080/metrics
# or check APM / Prometheus / Grafana
```

Checklist:

- ✓ Check application metrics
- ✓ Look for slow endpoints
- ✓ Check GC pauses (Java, Go, etc.)

8. CHECK NODE PRESSURE

```
$ kubectl describe node <node-name> | \
  grep -A5 "Conditions:"
$ kubectl top node
$ df -h
$ free -h
```

Checklist:

- ✓ Is node under memory pressure?
- ✓ Is disk space low?
- ✓ Is CPU on the node saturated?

9. OPTIMIZE APPLICATION

Application level

- Profile CPU and Memory
- Fix memory leaks
- Optimize queries
- Reduce unnecessary logs
- Improve caching strategy

- ✓ Profile and optimize code
- ✓ Reduce memory footprint
- ✓ Optimize heavy operations

10. RIGHT-SIZE RESOURCES

Use real metrics

- Set CPU requests ~ 50-70% of usage
- Set CPU limits ~ 2x requests
- Set Memory requests ~ 70% of usage
- Set Memory limits ~ 1.5x usage

- Right-size requests
- Set appropriate limits
- Avoid over-provisioning

11. RETEST AND MONITOR

```
$ kubectl top pod <pod-name> -n <ns>
$ watch -n 5 kubectl top pod <pod-name> -n <ns>
$ kubectl get events -n <ns>
```

- ✓ Monitor after changes
- ✓ Ensure no more OOMKills
- ✓ Confirm performance improved

12. PREVENT REOCCURRENCE

- Set requests and limits for all Pods
- Use HPA for CPU scaling
- Use VPA for recommendations
- Monitor with alerts
- Review app performance regularly

- ✓ Implement resource policies
- ✓ Set alerts for high usage
- ✓ Document and review

PREVENTION BEST PRACTICES

- Always set CPU and Memory requests and limits.
- Monitor resource usage with Prometheus/Grafana.
- Use HPA for CPU based scaling.
- Use VPA for right-sizing recommendations.
- Test under load before going to production.
- Optimize code and dependencies.

RESOURCE TUNING GUIDELINE

Resource	Requests (Rule of Thumb)	Limits (Rule of Thumb)
CPU	50-70% of avg usage	1.5x - 2x of requests
Memory	70% of avg usage	1.2x - 1.5x of requests

KEY TAKEAWAY

High CPU or Memory usage can slow down your app or crash Pods. Monitor, set proper requests and limits, optimize your application, and use autoscaling to keep your system healthy and performant.

VERIQTA

USEFUL COMMANDS

- kubectl top pods --all-namespaces
- kubectl top nodes
- kubectl describe pod <pod> -n <ns>
- kubectl get events -n <ns> --sort-by=.lastTimestamp'
- kubectl exec <pod> -n <ns> -- free -h
- kubectl exec <pod> -n <ns> -- ps aux --sort=-%mem
- kubectl describe node <node>
- kubectl get hpa -n <ns>
- kubectl get vpa -n <ns>
- kubectl logs <pod> -n <ns> --tail=100
- watch -n 5 kubectl top pods -n <ns>

VERIQTA

kubernetes

INCIDENT #12
HORIZONTAL POD AUTOSCALER NOT SCALING

HPA is configured, but the number of Pods stays the same even under high load.

AT A GLANCE

- Verify Metrics Server and metrics
- Check HPA status and events
- Validate CPU or custom metrics
- Review scaling policies and limits
- Fix configuration and test scaling

WHAT DOES IT MEAN?

The HPA is not increasing or decreasing the number of Pods as expected. This is usually due to missing metrics, misconfiguration, or scaling constraints. Without valid metrics, HPA cannot make scaling decisions.

STATUS FLOW



EXAMPLE: HPA STATUS

```
$ kubectl get hpa
NAME        REFERENCE           TARGETS   MINPODS  MAXPODS  REPLICAS  AGE
myapp-hpa   Deployment/myapp    120%/50%  2        10       2         15m

$ kubectl describe hpa myapp-hpa
Metrics:                    ( current / target )
  resource cpu              120% (240m) / 50%
Events:
  Type     Reason            Age   From                       Message
  Warning  FailedGetMetrics  2m    horizontal-pod-autoscaler
    unable to get metrics for resource cpu:
    no metrics returned from resource metrics API
```

HPA wants to scale, but it cannot get valid metrics or is blocked by configuration.

COMMON CAUSES

- Metrics Server is not installed or not healthy
- CPU metrics not available
- Custom metrics adapter not running
- Target metrics misconfigured
- Min/Max replicas set too low or too high
- Scaling policies restrict scaling
- Behavior settings too aggressive or strict
- Resource requests not set on containers
- RBAC permissions for metrics missing

1. CHECK HPA STATUS

```
$ kubectl get hpa
$ kubectl describe hpa <hpa-name>
$ kubectl get events --sort-by=.lastTimestamp
  | grep <hpa-name>
```

Checklist:
- ✓ Is the HPA active?
- ✓ Are there any warning events?
- ✓ What is the current metrics?

2. VERIFY METRICS SERVER

```
$ kubectl get apiservices | grep metrics
$ kubectl get deployment -n kube-system
  metrics-server
$ kubectl top nodes
$ kubectl top pods --all-namespaces
```

Checklist:
- ✓ Is Metrics Server available?
- ✓ Do top commands return metrics?
- ✓ Any errors in Metrics Server logs?

3. CHECK CPU METRICS

```
$ kubectl top pods -l app=myapp
$ kubectl describe pod <pod-name> |
  grep -i "cpu:"
$ kubectl get hpa <hpa-name> -o yaml
```

Checklist:
- ✓ Is CPU usage above target?
- ✓ Do Pods have CPU requests?
- ✓ Are units (m or %) correct?

4. CHECK CUSTOM METRICS

```
$ kubectl get --raw \
  /apis/custom.metrics.k8s.io/v1beta1
$ kubectl get hpa <hpa-name> -o yaml
```

Checklist:
- ✓ Is metrics adapter running?
- ✓ Are custom metrics available?
- ✓ Is metric name/label correct?

5. CHECK POD RESOURCE REQUESTS

```
$ kubectl get pod -l app=myapp -o yaml |
  grep -A5 -i resources:
$ kubectl explain pod.spec.containers.resources
```

Checklist:
- ✓ Do containers have CPU requests?
- ✓ HPA needs requests to calculate utilization.

6. CHECK SCALING BOUNDS

```
$ kubectl get hpa <hpa-name> -o yaml
$ kubectl get deploy myapp
```

Checklist:
- ✓ Are minReplicas and maxReplicas appropriate?
- ✓ Is current replica count at the max?

7. CHECK SCALING POLICIES

```
$ kubectl get hpa <hpa-name> -o yaml |
  grep -A10 "behavior:"
$ kubectl describe hpa <hpa-name>
```

Checklist:
- ✓ Are scaleUp/scaleDown policies too restrictive?
- ✓ Check stabilizationWindowSeconds.

8. CHECK TARGET METRICS

```
$ kubectl get hpa <hpa-name> -o yaml |
  grep -A5 "targetCPUUtilizationPercentage"
```

Checklist:
- ✓ Is target CPU % realistic?
- ✓ Too low = always scaling
- ✓ Too high = never scaling

9. CHECK WORKLOAD LOAD

```
$ kubectl exec -it <pod-name> --
  top
$ kubectl exec -it <pod-name> --
  ab -n 1000 -c 20 http://localhost
```

Checklist:
- ✓ Is the application actually under load?
- ✓ Verify load reaches the Pods.

10. CHECK EVENTS

```
$ kubectl get events --sort-by=.lastTimestamp
  | grep -i hpa
$ kubectl describe hpa <hpa-name>
```

Checklist:
- ✓ Look for FailedGetMetrics, FailedComputeMetrics, or BackOff events.

11. VALIDATE RBAC PERMISSIONS

```
$ kubectl auth can-i get --raw \
  /metrics.k8s.io --as=system:serviceaccount:
  kube-system:metrics-server"
```

Checklist:
- ✓ Does Metrics Server have correct permissions?
- ✓ Check ClusterRole and bindings.

12. RELOAD AND TEST SCALING

```
$ kubectl rollout restart deploy myapp
$ kubectl get hpa -w
```

Checklist:
- ✓ Generate load and watch scaling.
- ✓ Pods should scale up/down according to policies.

PREVENTION BEST PRACTICES

- Always install and monitor Metrics Server.
- Set proper resource requests for all containers.
- Use realistic target metrics.
- Define appropriate min/max replicas.
- Review and test scaling policies regularly.

HPA SCALING FLOW

Collect Metrics → Calculate Utilization → Compare with Target → Apply Scaling Policies → Adjust Pod Count → Observe & Stabilize

KEY TAKEAWAY

HPA depends on reliable metrics and correct configuration. Validate metrics, requests, targets, and policies first. Fix the root cause, then test with real load to ensure autoscaling works as expected.

VERIQTA

USEFUL COMMANDS

- kubectl get hpa
- kubectl describe hpa <name>
- kubectl get events --sort-by=.lastTimestamp'
- kubectl top pods
- kubectl top nodes
- kubectl get --raw /apis/custom.metrics.k8s.io/v1beta1
- kubectl auth can-i get --raw /metrics.k8s.io
- kubectl rollout restart deploy <name>
- watch -n 2 kubectl get hpa

kubernetes

VERIQTA

INCIDENT #13
NAMESPACE RESOURCE QUOTA EXCEEDED

Pods cannot be created or updated because the Namespace has reached its resource quota limits.

AT A GLANCE

- Check ResourceQuota and usage
- Review LimitRanges in the Namespace
- Validate failed Pod events
- Plan capacity and set realistic limits
- Design for multi-tenant environments

WHAT DOES IT MEAN?

A ResourceQuota restricts the total amount of resources that can be consumed in a Namespace.

When the quota is reached, new Pods, Deployments, or other resources cannot be created or scaled.

This protects the cluster from one team or workload consuming everything.

STATUS FLOW

Quota Configured → Resources Used → Quota Reached → Resource Creation Fails

✕ Pod/Resource Creation Blocked

EXAMPLE: QUOTA EXCEEDED

```
$ kubectl describe quota -n prod
Name:           prod-quota
Resource        Used    Hard
cpu             3800m   4000m
memory          7Gi     8Gi
pods            48      50
requests.cpu    3800m   4000m
requests.memory 7Gi     8Gi
Events:
  Type     Reason          Age   From                   Message
  Warning  FailedCreate    2m    replicaset-controller
  exceeded quota: prod-quota, requested: pods=2, used: pods=50,
  limited: pods=50
```

The Namespace "prod" has reached its Pod quota. New Pods cannot be created.

COMMON CAUSES

- ResourceQuota limits are too small
- Uncontrolled scaling of Deployments/Jobs
- Unexpected spikes in Pod count
- Memory or CPU limits reached
- Missing or misconfigured LimitRanges
- Test workloads left running
- No capacity planning for growth
- Poor governance in multi-tenant clusters

1. CHECK FAILED POD EVENTS

```
$ kubectl get events -n prod --sort-by=.lastTimestamp | grep -i exceeded
```

Checklist:

- ✓ Look for "exceeded quota" messages
- ✓ Identify which resource is blocked
- ✓ Note the exact quota name

2. DESCRIBE RESOURCE QUOTA

```
$ kubectl describe quota -n prod
```

Checklist:

- ✓ Review Used vs Hard for each resource
- ✓ Identify which resource is at the limit
- ✓ Check both object count and requests

3. CHECK NAMESPACE USAGE

```
$ kubectl top pod -n prod
```

Checklist:

- ✓ Review CPU and Memory usage
- ✓ Find workloads consuming the most
- ✓ Identify idle or orphaned resources

4. CHECK LIMITRANGES

```
$ kubectl describe limitrange -n prod
```

Checklist:

- ✓ Verify default requests and limits
- ✓ Ensure values are reasonable
- ✓ Avoid very low or zero defaults

5. LIST NAMESPACE RESOURCES

```
$ kubectl get all -n prod
```

Checklist:

- ✓ Check number of Pods
- ✓ Check number of Services, PVCs, etc.
- ✓ Remove unused or old resources

6. CHECK CONTROLLERS

```
$ kubectl get deploy,sts,ds,job,cronjob -n prod
```

Checklist:

- ✓ Look for over-provisioned replicas
- ✓ Identify Jobs creating many Pods
- ✓ Review CronJobs and schedules

7. SCALE DOWN OR CLEAN UP

```
$ kubectl scale deploy <name> -n prod --replicas=2
```

Checklist:

- ✓ Scale down non-critical workloads
- ✓ Delete completed Jobs
- ✓ Remove unused resources

8. UPDATE RESOURCE QUOTA

```
$ kubectl edit quota prod-quota -n prod
```

Checklist:

- ✓ Increase limits if cluster capacity allows
- ✓ Adjust based on real usage
- ✓ Be cautious and plan capacity

9. PLAN CAPACITY

```
$ kubectl top nodes
```

Checklist:

- ✓ Check node CPU and Memory
- ✓ Ensure enough free capacity
- ✓ Plan for future growth

10. SET ALERTS

```
$ kubectl get --raw /metrics | grep quota
$ # Use Prometheus alert for quota usage
```

Checklist:

- ✓ Monitor quota usage %
- ✓ Alert at 70% or 80%
- ✓ Avoid reaching 100%

11. ENFORCE GOVERNANCE

- Define quotas per Namespace
- Define LimitRanges
- Review workloads regularly
- Educate teams on usage

12. DOCUMENT & REVIEW

Checklist:

- ✓ Document quotas and limits
- ✓ Review quarterly
- ✓ Adjust based on business needs

PREVENTION BEST PRACTICES

- Set realistic ResourceQuota per Namespace
- Use LimitRanges for default requests/limits
- Monitor usage continuously
- Review and clean up unused resources
- Plan capacity before scaling workloads

QUOTA COMPONENTS

Object Count (Pods, PVCs) + CPU (Limit/Request) + Memory (Limit/Request) + Storage (PVCs) → Other (Services, etc.)

All components contribute to the total quota usage.

KEY TAKEAWAY

Resource quotas protect the cluster and ensure fair usage across teams. Design quotas and limits carefully, monitor usage, and plan capacity for sustainable growth.

VERIQTA

USEFUL COMMANDS

- kubectl describe quota -n <ns>
- kubectl get quota -A
- kubectl get events -n <ns>
- kubectl top pod -n <ns>
- kubectl describe limitrange -n <ns>
- kubectl get all -n <ns>
- kubectl get deploy,sts,job -n <ns>
- kubectl top nodes
- kubectl edit quota <name> -n <ns>
- kubectl scale deploy <name> -n <ns> --replicas=<n>
- kubectl get --raw /metrics | grep quota

kubernetes

VERIQTA

INCIDENT #14
NETWORKPOLICY BLOCKING APPLICATION TRAFFIC

NetworkPolicy rules are blocking traffic between Pods or from external clients.

AT A GLANCE

- Validate NetworkPolicies and rules
- Check allowed ingress and egress
- Test Pod-to-Pod communication
- Verify Namespace isolation rules
- Approve only required traffic

WHAT DOES IT MEAN?

A NetworkPolicy restricts how Pods communicate with other Pods, Namespaces, or external endpoints.

If traffic is not explicitly allowed, it will be denied.

This is by design and helps secure the cluster, but misconfigurations can break applications.

STATUS FLOW

Traffic Works → Policy Applied → Traffic Blocked → Validation Fails → Fix Policy

✕ Communication Blocked

EXAMPLE: TRAFFIC BLOCKED

```
$ kubectl describe networkpolicy deny-all -n prod
Name:         deny-all
Namespace:    prod
PodSelector:  <none>
PolicyTypes:  Ingress, Egress
Ingress:      <none>
Egress:       <none>
Age:          10m
$ kubectl exec -it pod-a -n prod -- curl http://pod-b:8080
curl: (7) Failed to connect to pod-b port 8080: Connection refused
```

No ingress rule allows traffic to pod-b.
Traffic is blocked by default deny policy.

COMMON CAUSES

- Default deny policy without allow rules
- Missing ingress rule for the application
- Missing egress rule for external calls
- Incorrect namespace or podSelector
- Block to DNS or kube-dns/CoreDNS
- Port or protocol mismatch
- Overly restrictive rules
- Policy applied to wrong Pods

1. LIST NETWORKPOLICIES

```
$ kubectl get networkpolicies -A
$ kubectl get networkpolicies -n <ns>
```

Checklist:
- ✓ Identify policies in the Namespace
- ✓ Note policies with default deny
- ✓ Verify which Pods are selected

2. DESCRIBE POLICY

```
$ kubectl describe networkpolicy <name> -n <ns>
```

Checklist:
- ✓ Review podSelector
- ✓ Review ingress rules
- ✓ Review ingress rules
- ✓ Check policyTypes

3. CHECK POD LABELS

```
$ kubectl get pods -n <ns> --show-labels
```

Checklist:
- ✓ Do Pod labels match podSelector?
- ✓ Missing label = rule not applied
- ✓ Extra labels = unexpected match

4. TEST POD-TO-POD CONNECTIVITY

```
$ kubectl exec -it pod-a -n <ns> -- sh
# curl http://pod-b:8080/health
```

Checklist:
- ✓ Test from source Pod
- ✓ Use service name or Pod IP
- ✓ Verify port and protocol

5. CHECK EGRESS RULES

```
$ kubectl describe networkpolicy <name>
# Look under Egress section
```

Checklist:
- ✓ Are external calls allowed?
- ✓ DNS, API, DB, Queue?
- ✓ Missing egress = outbound blocked

6. CHECK DNS ACCESS

```
$ kubectl exec -it <pod> -n <ns> -- nslookup
kubernetes.default.svc
```

Checklist:
- ✓ DNS blocked breaks many apps
- ✓ Allow UDP/TCP 53 to CoreDNS
- ✓ Without DNS, services fail

DNS

7. CHECK NAMESPACE ISOLATION

```
$ kubectl get networkpolicy -A
# Verify cross-namespace rules
```

Checklist:
- ✓ Are you blocking other Namespaces?
- ✓ Ensure only required isolation
- ✓ Validate allowed communication

8. VALIDATE POLICY EFFECT

```
$ kubectl get networkpolicy <name> -n <ns> -o yaml
```

Checklist:
- ✓ Review final YAML
- ✓ Look for missing allow rules
- ✓ Check ports and protocols

9. ALLOW REQUIRED TRAFFIC

```
$ kubectl apply -f allow-app-traffic.yaml
```

- ✓ Add specific allow rules
- ✓ Least privilege principle
- ✓ Allow only required ports

10. RETEST CONNECTIVITY

```
$ kubectl exec -it pod-a -n <ns> --
curl http://pod-b:8080/health
```

- ✓ Test again after fix
- ✓ Validate from all directions
- ✓ Confirm success

11. MONITOR AND LOGS

```
$ kubectl logs <pod> -n <ns>
# Check for denied connections
```

- ✓ Monitor application logs
- ✓ Watch for dropped connections
- ✓ Use CNI or Calico logs if needed

12. DOCUMENT AND REVIEW

- ✓ Document policy intent
- ✓ Review with team
- ✓ Update runbooks
- ✓ Re-audit regularly

PREVENTION BEST PRACTICES

- Start with default allow, then restrict
- Use specific podSelectors
- Allow DNS, monitoring, and logging
- Test policies before enforcing cluster-wide
- Review policies after every change

NETWORKPOLICY TRAFFIC FLOW

Source Pod → NetworkPolicy Evaluation → Allowed Traffic / Blocked Traffic → Destination Pod

If not explicitly allowed, traffic is denied.

KEY TAKEAWAY

NetworkPolicies are powerful but easy to misconfigure. Always validate rules, test Pod communication, and allow only the traffic your application requires. Secure by default, but verify continuously.

VERIQTA

USEFUL COMMANDS

- kubectl get networkpolicies -A
- kubectl describe networkpolicy <name> -n <ns>
- kubectl get pods -n <ns> --show-labels
- kubectl exec -it <pod> -n <ns> -- sh
- kubectl exec -it <pod> -n <ns> -- curl http://<service>:<port>
- kubectl exec -it <pod> -n <ns> -- nslookup kubernetes.default.svc
- kubectl get networkpolicy <name> -n <ns> -o yaml
- kubectl apply -f <file.yaml>
- kubectl logs <pod> -n <ns>
- kubectl get events -n <ns> --sort-by='.lastTimestamp'
- kubectl describe pod <pod> -n <ns>

VERIQTA

kubernetes

INCIDENT #15

API SERVER PERFORMANCE DEGRADATION

The Kubernetes API Server is slow or unresponsive, affecting the entire cluster.

AT A GLANCE

- Check API Server latency and health
- Validate etcd performance and status
- Review control plane resource usage
- Identify long running or excessive requests
- Recover and restore cluster stability

WHAT DOES IT MEAN?

The API Server is the front door of the cluster. When it is slow, all kubernetes operations, controllers, scheduling, and automation are affected.

Root causes are often linked to etcd performance, high load, or control plane resource pressure.

STATUS FLOW

Healthy → High Latency → Requests Lagging → API Unresponsive

✕ Cluster Impact

EXAMPLE: API SERVER SLOW

```
$ kubectl get --raw='/readyz?verbose'
etcd.....ok
etcd has leader.....ok
livez.....ok
readyz.....fail

$ kubectl get --raw='/metrics' | grep apiserver_request_duration_seconds
apiserver_request_duration_seconds_bucket{le="1"}   1023
apiserver_request_duration_seconds_bucket{le="5"}   8421
apiserver_request_duration_seconds_bucket{le="30"}  12045
apiserver_request_duration_seconds_bucket{le="60"}  14502
```

High latency or failing readiness indicates API Server or etcd issues.

COMMON CAUSES

- etcd slow or under heavy load
- High number of API requests
- Large number of objects (e.g., Pods, CRDs)
- Long running or stuck requests
- Insufficient CPU or memory on control plane
- Disk I/O issues on control plane
- Network latency between API Server and etcd
- Compromised or overloaded control plane node
- Misconfigured admission webhooks

1. CHECK API SERVER HEALTH

```
$ kubectl get --raw='/livez'
$ kubectl get --raw='/readyz?verbose'
$ kubectl get componentstatuses
```

Checklist:
- ✓ API livez returns ok
- ✓ API readyz returns ok
- ✓ No failing dependencies

2. CHECK API SERVER LATENCY

```
$ time kubectl get pods -A
$ kubectl get --raw='/metrics' \
  grep apiserver_request_duration_seconds
```

Checklist:
- ✓ Latency < 1s is healthy
- ✓ P95 latency should be low
- ✓ Watch for high duration buckets

3. CHECK etcd HEALTH

```
$ kubectl exec -n kube-system \
  -it etcd-$(hostname) -- sh -c \
  "etcdctl endpoint health --write-out=table"
```

Checklist:
- ✓ All etcd endpoints healthy
- ✓ Leader is stable
- ✓ No high failed proposals

4. CHECK etcd PERFORMANCE

```
$ kubectl exec -n kube-system \
  -it etcd-$(hostname) -- sh -c \
  "etcdctl endpoint status --write-out=table"
```

Checklist:
- ✓ DB size is reasonable
- ✓ Has Leader = true
- ✓ DB size not growing abnormally

5. CHECK CONTROL PLANE RESOURCES

```
$ top -u
$ free -h
$ df -h
```

Checklist:
- ✓ CPU usage not maxed
- ✓ Memory usage healthy
- ✓ Disk not full

6. CHECK API SERVER POD/PROCESS

```
$ kubectl -n kube-system get pods -l \
  component=kube-apiserver -o wide
$ ps -ef | grep kube-apiserver
```

Checklist:
- ✓ API Server pod running
- ✓ No restarts or OOMKills
- ✓ Process using expected flags

7. CHECK API SERVER FLAGS

```
$ ps -ef | grep kube-apiserver
# Look for: --max-requests-inflight
# --max-mutating-requests-inflight
```

Checklist:
- ✓ Max inflight values tuned
- ✓ Watch/Read timeout configured
- ✓ Audit logging not overloading

8. CHECK REQUEST RATE

```
$ kubectl get --raw='/metrics' \
  | grep apiserver_request_total
```

Checklist:
- ✓ Request rate within limits
- ✓ Watch for sudden spikes
- ✓ Identify noisy clients

9. CHECK SLOW REQUESTS

```
$ kubectl get --raw='/debug/pprof/
  profile?seconds=30' > profile.out
$ Analyze with go tool pprof
```

Checklist:
- ✓ Identify slow handlers
- ✓ Check heavy list/watch ops
- ✓ Look for stuck requests

10. CHECK ADMISSION WEBHOOKS

```
$ kubectl get validatingwebhookconfigurations
$ kubectl get mutatingwebhookconfigurations
```

Checklist:
- ✓ Webhook endpoints healthy
- ✓ Timeouts configured properly
- ✓ No failing webhook calls

11. CHECK NETWORK TO etcd

```
$ kubectl exec -n kube-system \
  -it kube-apiserver-xxxxx -- sh
$ nc -zv etcd 2379
```

Checklist:
- ✓ Network latency is low
- ✓ No packet loss
- ✓ TLS certificates valid

12. CHECK EVENT LOGS

```
$ kubectl -n kube-system logs \
  -l component=kube-apiserver --tail=100
```

Checklist:
- ✓ Look for timeouts
- ✓ etcd errors or slow ops
- ✓ Admission webhook errors

PREVENTION BEST PRACTICES

- Monitor API Server and etcd metrics
- Set right-size control plane resources
- Limit excessive list/watch operations
- Use pagination and field selectors
- Tune max inflight requests
- Keep etcd healthy and backed up
- Regularly review admission webhooks

API SERVER RECOVERY WORKFLOW

Detect Degradation → Identify Root Cause → Stabilize etcd → Free Resources → Tweak Config → Validate & Monitor

Recover, verify stability, and monitor continuously.

KEY TAKEAWAY

A slow API Server affects everything. Monitor latency, protect etcd, control request load, and right-size your control plane. A healthy control plane is the foundation of a healthy Kubernetes cluster.

VERIQTA

USEFUL COMMANDS

- kubectl get --raw='/livez'
- kubectl get --raw='/readyz?verbose'
- kubectl get --raw='/metrics'
- kubectl tgt componentstatuses
- etcdctl endpoint health --write-out=table
- etcdctl endpoint status --write-out=table
- kubectl get events -A --sort-by=.lastTimestamp
- kubectl top pods -n kube-system
- kubectl logs -n kube-system -l component=kube-apiserver
- kubectl get --raw='/debug/pprof/profile?seconds=30'
- curl -k https://localhost:6443/healthz

VERIQTA

SENIOR KUBERNETES INCIDENT INVESTIGATION WORKFLOW

A repeatable, structured approach senior engineers use to investigate, resolve, and prevent Kubernetes incidents.

AT A GLANCE

- Triage the incident quickly
- Collect the right evidence
- Identify the real root cause
- Recover the service safely
- Learn and prevent recurrence

1 INCIDENT TRIAGE

- Detect and acknowledge
- Understand impact
- Identify affected services/workloads
- Determine severity
- Communicate early

Goal: Stop the noise, focus on what matters.

2 EVIDENCE COLLECTION

- Capture cluster state
- Collect logs and events
- Gather metrics
- Save configurations
- Preserve environment snapshot

Goal: Build a complete picture before acting.

3 ROOT CAUSE ANALYSIS

- Analyze symptoms
- Correlate evidence
- Validate hypotheses
- Identify the true underlying cause
- Document findings

Goal: Find the real cause, not just the obvious.

4 RECOVERY WORKFLOW

- Choose recovery strategy
- Execute fix/rollback
- Validate recovery
- Monitor closely
- Ensure full service restoration

Goal: Restore service safely and completely.

5 POST-INCIDENT REVIEW

- Review timeline
- Identify gaps
- Update runbooks
- Improve monitoring/alerts
- Share knowledge

Goal: Learn, improve, and prevent recurrence.

KEY PRINCIPLES

Stay Calm
Don't guess. Follow the process.

Collect First
Understand before you change anything.

Prove, Don't Assume
Use evidence to confirm what is really happening.

Minimize Impact
Fix safely and validate before closing.

Document Everything
Your future self and team will thank you.

DETAILED WORKFLOW

1. INCIDENT TRIAGE

Checklist:
- ✓ Validate the alert
- ✓ Check user impact
- ✓ Identify affected namespace, pod, service, node
- ✓ Check blast radius
- ✓ Communicate in team channel

Useful Commands:

```
kubectl get pods -A
kubectl get nodes
kubectl get events -A --sort-by=.lastTimestamp
kubectl get svc -A
```

Focus:
What is broken?
Who is affected?
How bad is it?

2. EVIDENCE COLLECTION

Checklist:
- ✓ Capture cluster state
- ✓ Collect logs and events
- ✓ Check metrics
- ✓ Save configs and manifests
- ✓ Take a snapshot (if possible)

Useful Commands:

```
kubectl get pods -A -o wide
kubectl get events -A --sort-by=.lastTimestamp > events.log
kubectl describe pod <pod> -n <ns>
kubectl logs <pod> -n <ns> --previous > pod.log
kubectl top pods -A
kubectl get all -n <ns> -o yaml > state-<date>.yaml
```

Focus:
Get facts.
Build the evidence.

3. ROOT CAUSE ANALYSIS

Checklist:
- ✓ Review all evidence
- ✓ Correlate events, logs, metrics, and configs
- ✓ Validate assumptions
- ✓ Find the underlying cause
- ✓ Document the root cause

Useful Commands:

```
kubectl describe <resource> -n <ns>
kubectl logs <pod> -n <ns>
kubectl get events -A | grep -i error
kubectl get nodes
kubectl top nodes
```

Focus:
Why did this happen?
What triggered it?
What failed first?

4. RECOVERY WORKFLOW

Checklist:
- ✓ Decide recovery approach
- ✓ Execute fix or rollback
- ✓ Verify service is healthy
- ✓ Monitor for stability
- ✓ Close incident only after full validation

Useful Commands:

```
kubectl rollout undo deploy/<name> -n <ns>
kubectl rollout status deploy/<name> -n <ns>
kubectl delete pod <pod> -n <ns>
kubectl scale deploy/<name> -n <ns> --replicas=2
```

Focus:
Restore service.
Validate fully.
Monitor closely.

5. POST-INCIDENT REVIEW

Checklist:
- ✓ Review full timeline
- ✓ Identify what went well
- ✓ Identify what failed
- ✓ Update runbooks/playbooks
- ✓ Improve alerts and dashboards
- ✓ Share knowledge

Useful Actions:
- Update documentation
- Improve monitoring
- Add tests or validations
- Conduct blameless review
- Track action items

Focus:
Learn from it.
Make the next incident easier to solve.

EVIDENCE TO ALWAYS COLLECT

- Pod logs (current & previous)
- Events (cluster-wide)
- Resource descriptions
- Metrics (CPU, Memory, Disk, Network)
- Node status and conditions
- ConfigMaps, Secrets, and related configs

SEVERITY GUIDE

SEV-1	Critical	Complete outage, major impact
SEV-2	High	Partial outage, high impact
SEV-3	Medium	Degraded performance
SEV-4	Low	Minor issue, no user impact

GOLDEN RULE

Never make changes without understanding the problem.

Collect → Analyze → Understand → Then Act.

INVESTIGATION TIMELINE EXAMPLE

Alert	Triage	Evidence Collection	Root Cause Found	Recovery	Service Restored	Post Review
00:00	00:02	00:05	00:20	00:30	00:40	01:00+

REMEMBER

Incidents are inevitable.
Good process is optional.
Great engineers follow a process every time.
That is what separates senior engineers.

VERIQTA

KUBERNETES PRODUCTION ARCHITECTURE BEST PRACTICES

Design resilient, secure, scalable, and observable Kubernetes platforms that survive failure and scale with confidence.

AT A GLANCE

- Highly available control plane
- Multi-AZ resilient architecture
- Secure by default
- Observable and monitored
- Scalable and upgrade ready
- Disaster recovery prepared

1. HIGH AVAILABILITY CONTROL PLANE

- ✓ Run at least 3 API server replicas
- ✓ Distribute across 3 Availability Zones
- ✓ Use managed etcd with Multi-AZ
- ✓ Enable API server load balancing
- ✓ Enable automatic backup for etcd
- ✓ Regular upgrade and patching
- ✓ Enable audit logging
- ✓ Monitor API server and etcd health

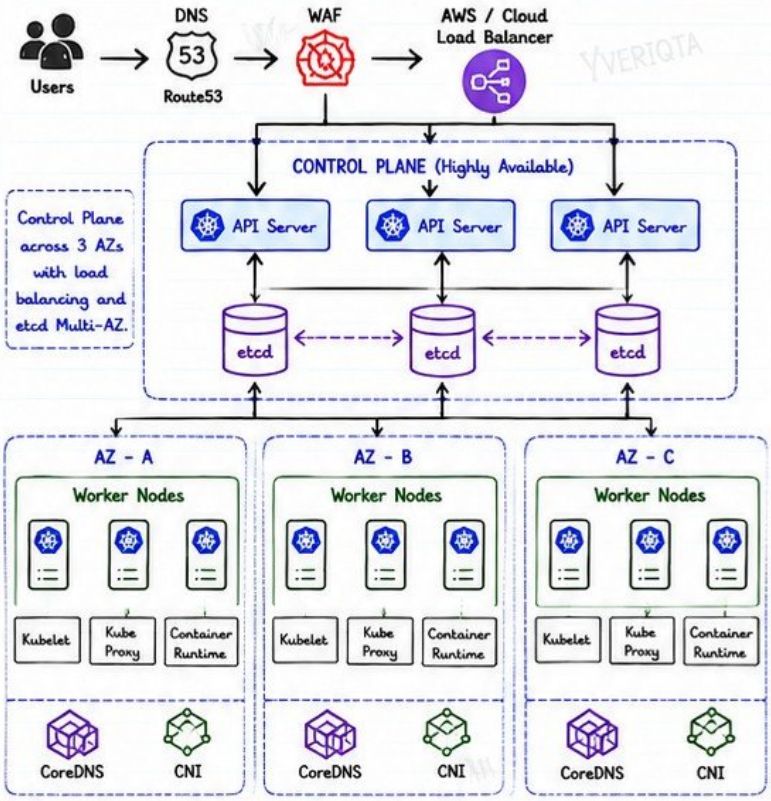
Goal: Keep the control plane available even during failures.

2. WORKER NODES

- ✓ Spread nodes across multiple AZs
- ✓ Use Auto Scaling Groups
- ✓ Right-size instances for workloads
- ✓ Use spot + on-demand mix
- ✓ Enable node taints and tolerations
- ✓ Keep nodes patched and hardened
- ✓ Enable kubelet read-only port
- ✓ Monitor node health and capacity

Goal: Maintain capacity, performance and node resiliency.

PRODUCTION ARCHITECTURE OVERVIEW



Observability

- Prometheus
- Grafana
- Alertmanager
- Loki / Logs
- Jaeger / Tracing

Centralized monitoring, alerting and tracing.

Security

- RBAC
- Network Policies
- Pod Security Standards
- Image Scanning
- Secrets Management
- Audit Logging

Security built in at every layer.

3. MULTI-AZ CLUSTERS

- ✓ Deploy control plane across 3 AZs
- ✓ Deploy worker nodes across 3+ AZs
- ✓ Use AWS Load Balancer across AZs
- ✓ Use cross-zone load balancing
- ✓ Place critical workloads across AZs
- ✓ Test AZ failure and recovery

Goal: Ensure high availability and fault tolerance.

4. MONITORING

- ✓ Monitor cluster, nodes, pods, and apps
- ✓ Use Prometheus for metrics
- ✓ Grafana for visualization
- ✓ Alertmanager for alert routing
- ✓ Centralized logs with Loki/ELK
- ✓ Distributed tracing with Jaeger
- ✓ Set SLOs and error budgets
- ✓ Proactive alerting and dashboards

Goal: Full visibility and faster incident response.

5. SECURITY

- ✓ Enable RBAC and least privilege
- ✓ Apply NetworkPolicies
- ✓ Use Pod Security Standards
- ✓ Scan container images regularly
- ✓ Store secrets in AWS Secrets Manager
- ✓ Enable audit logging
- ✓ Encrypt data in transit and at rest
- ✓ Regular vulnerability assessments

Goal: Secure by default, defend in depth.

6. STORAGE ARCHITECTURE

- ✓ Use StorageClass for all storage
- ✓ Use encrypted EBS/EFS/FSx volumes
- ✓ Enable snapshots and backups
- ✓ Use ReadWriteMany where needed
- ✓ Separate storage for stateful workloads
- ✓ Test restore and disaster recovery
- ✓ Monitor storage performance and usage

Goal: Durable, performant and resilient persistent storage.

BEST PRACTICES CHECKLIST

- Multi-AZ control plane
- etcd Multi-AZ with backups
- Nodes across multiple AZs
- Auto Scaling enabled
- Monitoring and alerting in place
- RBAC and Network Policies enabled
- Secrets managed securely
- Backups and DR tested
- Regular updates and patching
- Capacity planning and scaling tested

KEY PRINCIPLES

- Resilience over single points of failure
- Design for failure, test recovery
- Observe everything
- Automate scaling and recovery
- Backup everything, always

REMEMBER

- ☆ Architect for failure.
- ☆ Automate recovery.
- ☆ Monitor everything.
- ☆ Secure by default.
- ☆ Test continuously.
- ☆ Document and iterate.

kubernetes

VERIQTA

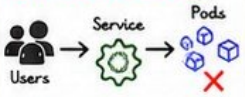
SENIOR KUBERNETES INTERVIEW SCENARIOS

Real production troubleshooting questions, expected senior-level answers and what interviewers look for.

AT A GLANCE

- Real production scenarios
- Senior-level troubleshooting
- Architecture discussions
- What interviewers look for
- How to think and communicate

SCENARIO (PRODUCTION)	TROUBLESHOOTING QUESTIONS	EXPECTED SENIOR-LEVEL ANSWER (SUMMARY)	WHAT INTERVIEWERS LOOK FOR	ARCHITECTURE / DISCUSSION POINTS

1. PODS ARE RUNNING BUT SERVICE IS DOWN
 
Symptoms:
• Pods are Running
• Service not reachable
• Application timeout
	• How do you start debugging?
	• What components will you check?
	• How do you isolate the issue?
	• What commands would you use?
	`$ kubectl get svc,pods -o wide`
	`$ kubectl describe svc <svc>`
	`$ kubectl get endpoints <svc>`
	`$ kubectl logs <pod> -n`
	`$ kubectl exec -it <pod> -- curl <svc>:<port>`
		• Check Service -> Endpoints -> Pods
		• Verify Endpoints exist and are healthy
		• Check NetworkPolicy, Ingress, DNS
		• Validate Pod readiness & logs
		• Test connectivity from inside cluster
		• Identify root cause and fix
		☆ Think in layers: Client -> DNS -> LB/Ingress -> Service -> Endpoints -> Pod -> App
			• Systematic approach
			• Kubernetes networking
			• Use of right commands
			• Ability to isolate issue
			• Clear communication
			• Understanding of components
				• Service discovery flow
				• Ingress vs Service
				• NetworkPolicy impact
				• DNS resolution
				• Health checks & probes
				• Observability stack

2. POD IN CRASHLOOPBACKOFF
 
Symptoms:
• Pod restarts continuously
• Application unavailable
	• What could cause this?
	• How do you investigate?
	• How do you fix it?
	`$ kubectl describe pod <pod>`
	`$ kubectl logs <pod> --previous`
	`$ kubectl get events --sort-by=.lastTimestamp`
	`$ kubectl exec -it <pod> -- sh`
		• Check logs (previous container logs)
		• Verify config, env, secret, volume mounts
		• Check resource limits / OOMKilled
		• Validate liveness/readiness probes
		• Reproduce and fix the root cause
		• Rollout restart or redeploy
		Look beyond the pod: config, code, dependencies, resources.
			• Root cause mindset
			• Knowledge of probes
			• Resource troubleshooting
			• Log analysis skills
			• Aware of common pitfalls
				• Probe configuration
				• Resource requests/limits
				• Secrets & ConfigMaps
				• Init containers impact
				• Rolling updates strategy

3. DEPLOYMENT ROLLOUT FAILED
 
Symptoms:
• New version not available
• Pods not updated
	• Why did the rollout fail?
	• How do you investigate?
	• How do you rollback safely?
	`$ kubectl rollout status deploy/<deploy>`
	`$ kubectl rollout history deploy/<deploy>`
	`$ kubectl describe rs <rs>`
	`$ kubectl logs <pod>`
	`$ kubectl rollout undo deploy/<deploy>`
		• Check ReplicaSet events and pod status
		• Validate image, resources, probes
		• Check quota, limits, node capacity
		• Use rollout history to rollback
		• Fix and re-deploy
		Roll forward if possible, rollback if necessary.
			• Understanding rollout
			• Use of history & undo
			• Safe deployment practices
			• Impact analysis
			• Minimize downtime
				• RollingUpdate strategy
				• MaxUnavailable/MaxSurge
				• Blue/Green or Canary
				• Image versioning
				• CI/CD integration

4. HIGH CPU/MEMORY USAGE ON NODES
 
Symptoms:
• Node CPU/Memory 90%+
• Pods throttled / OOMKilled
	• How do you identify the cause?
	• What do you check first?
	• How do you resolve it?
	`$ kubectl top nodes`
	`$ kubectl top pods -A --sort-by=cpu`
	`$ kubectl describe node <node>`
	`$ kubectl get events --all-namespaces`
		• Identify top consuming pods/containers
		• Check limits, requests, HPA configs
		• Review application performance
		• Add resources or optimize app
		• Use HPA/VPA for right sizing
		Balance: requests, limits, autoscaling and optimization.
			• Performance debugging
			• Metrics driven decisions
			• Autoscaling knowledge
			• Cost & efficiency mindset
			• Proactive monitoring
				• HPA / VPA
				• Resource quotas
				• Node autoscaling
				• Cluster autoscaler
				• Capacity planning

5. API SERVER UNRESPONSIVE
 
Symptoms:
• kubectl timeout
• Cluster dashboard down
	• What could cause this?
	• How do you verify?
	• How do you recover?
	`$ kubectl get --raw='/readyz?verbose'`
	`$ kubectl get --raw='/livez'`
	`$ kubectl top nodes`
	`$ etcdctl endpoint health`
	`$ journalctl -u kube-apiserver -f`
		• Check control plane resource usage
		• Validate etcd health & latency
		• Check API server logs & audit logs
		• Restart API server if needed
		• Restore from backup if necessary
		Protect etcd, it is the heart of Kubernetes.
			• Control plane knowledge
			• etcd understanding
			• Troubleshooting skills
			• Recovery strategies
			• Risk assessment
				• HA control plane
				• etcd backups
				• Resource isolation
				• Monitoring & alerting
				• Disaster recovery

SENIOR ENGINEER MINDSET



- ✓ Think from user impact backwards
- ✓ Validate assumptions with data
- ✓ Use observability and metrics
- ✓ Automate and document
- ✓ Prevent recurrence

KUBERNETES COMPONENTS YOU MUST KNOW



COMMUNICATION TIPS

- ✓ Clarify the problem
- ✓ Walk through your thought process
- ✓ Explain trade-offs
- ✓ Be concise and structured
- ✓ Always end with how you prevent it



USEFUL COMMANDS CHEAT SHEET



- `kubectl get pods -A -o wide`
- `kubectl describe pod <pod>`
- `kubectl logs <pod> --previous`
- `kubectl get events -A --sort-by=.lastTimestamp`
- `kubectl top pods -A --sort-by=cpu`
- `kubectl get endpoints <svc>`
- `kubectl get networkpolicies -A`
- `kubectl get ingress -A`
- `kubectl rollout status deploy/<deploy>`
- `kubectl rollout history deploy/<deploy>`
- `kubectl get rs,deploy -A`
- `kubectl get all -A`

REMEMBER



- It's not about memorizing commands.
- It's about how you think, troubleshoot and solve real production problems.

Think like a SRE. Act like an Owner.
That's what we look for.

VERIQTA

KUBERNETES PRODUCTION READINESS CHECKLIST

Use this checklist to ensure your Kubernetes platform is secure, reliable, observable, and ready for production scale.

AT A GLANCE

- Production-ready Kubernetes clusters
- Secure by default
- Highly available and resilient
- Observable and operable
- Scalable and performance optimized
- Prepared for incidents and recovery

1. CLUSTER CHECKLIST

- [ ] Kubernetes version is supported
- [ ] All control plane components healthy
- [ ] etcd is healthy and backed up
- [ ] Nodes are in Ready state
- [ ] No critical events in the cluster
- [ ] CoreDNS is healthy and redundant
- [ ] Cluster time is synchronized (NTP)
- [ ] Resource requests and limits defined
- [ ] Namespaces are organized and labeled
- [ ] kubectl access and RBAC validated
- [ ] Cluster autoscaler enabled
- [ ] Pod disruption budgets configured

Goal: A stable, healthy, and well-maintained cluster.

2. SECURITY CHECKLIST

- [ ] RBAC is enabled and least privilege enforced
- [ ] ServiceAccounts are scoped and reviewed
- [ ] PodSecurityStandards enforced
- [ ] NetworkPolicies applied to namespaces
- [ ] ImagePullSecrets used for private registries
- [ ] Container images scanned for vulnerabilities
- [ ] Secrets encrypted (at rest and in transit)
- [ ] etcd encryption enabled
- [ ] Audit logging enabled and shipped
- [ ] Admission controllers enabled (OPA/Gatekeeper)
- [ ] Regular security patching process in place
- [ ] Node hardening and CIS benchmarks applied

Goal: Secure the cluster, workloads, and data.

3. NETWORKING CHECKLIST

- [ ] CNI plugin installed and healthy
- [ ] Pod-to-pod communication validated
- [ ] NetworkPolicies applied and tested
- [ ] Ingress controller is HA and monitored
- [ ] TLS certificates valid and auto-renewing
- [ ] DNS resolution works inside the cluster
- [ ] Service types validated (ClusterIP/NodePort/LB)
- [ ] External access only through Ingress/ALB
- [ ] Egress rules controlled and monitored
- [ ] Network latency and packet loss checked
- [ ] Load balancing and failover tested
- [ ] IP address management documented

Goal: Reliable, secure, and observable networking.

4. STORAGE CHECKLIST

- [ ] StorageClass defined and default set
- [ ] PersistentVolumes provision automatically
- [ ] PV reclaim policy reviewed
- [ ] ReadWriteMany / access modes validated
- [ ] Volume snapshots enabled and tested
- [ ] Backup and restore process validated
- [ ] etcd backups scheduled and tested
- [ ] Application data backup strategy defined
- [ ] Storage capacity and IOPS monitored
- [ ] No orphaned PVs or unbound claims
- [ ] CSI drivers healthy and up to date
- [ ] Disaster recovery storage plan documented

Goal: Durable, performant, and recoverable storage.

5. MONITORING CHECKLIST

- [ ] Metrics Server is installed and healthy
- [ ] Prometheus is running and scraping targets
- [ ] Grafana dashboards are configured
- [ ] Alertmanager is configured with alerts
- [ ] Node, pod, and cluster metrics monitored
- [ ] Logs collected (EFK / Loki / CloudWatch)
- [ ] Audit logs collected and retained
- [ ] SLOs / SLIs defined for critical services
- [ ] Alerts tested and notification working
- [ ] Dashboards shared with the team
- [ ] Capacity monitoring and forecasting enabled
- [ ] Synthetic monitoring for user workflows

Goal: Full visibility and proactive alerting.

6. HIGH AVAILABILITY CHECKLIST

- [ ] Control plane runs on 3+ nodes / AZs
- [ ] etcd cluster has odd number of members
- [ ] Worker nodes spread across AZs
- [ ] Multi-AZ LoadBalancer or Ingress configured
- [ ] Pod disruption budgets configured
- [ ] Critical workloads are replicated (HA)
- [ ] Cluster autoscaler configured
- [ ] Node health monitoring and auto-repair
- [ ] Regular backups of etcd and configurations
- [ ] Disaster recovery plan tested
- [ ] Failover and recovery runbooks documented
- [ ] Chaos testing / game days executed

Goal: Survive failures and recover quickly.

FINAL VERIQTA PRODUCTION PRINCIPLES

RELIABILITY
Build for failure. Assume something will break. Plan for it.

SECURITY
Secure by default. Least privilege everywhere.

OBSERVABILITY
You can't fix what you can't see. Monitor everything that matters.

PERFORMANCE
Right-size resources. Continuously optimize. Measure and improve.

AUTOMATION
Automate everything repeatable. Reduce manual intervention.

COLLABORATION
Document, share, review, and iterate as a team.

RESILIENCE
Design for scale. Design for chaos. Recover with confidence.

A production-ready Kubernetes cluster is not about features, it's about discipline, observability, security, automation, and continuous improvement.

REMEMBER

- Validate regularly
- Document everything
- Test failures
- Trust but verify